

RAPPORT DE PROJET

Détection de Fraude Bancaire par Machine Learning Pipeline MLOps/DataOps de bout en bout

Livrable 7 — Rapport final

*Module MLOps & DataOps — Pr. Mohammed AIT DAOUD, FSBM, Université Hassan II de Casablanca
Master 1ère Année — Science des Données et Intelligence Artificielle*

Année Universitaire 2025 — 2026

Réalisé par :

- *Anass Dabibe*
- *Hassan El Hadi*
- *Yassin Farih*
- *Adnane Dahbi*
- *Aymane El Badri*
- *Sifeddine Legnioui*
- *Youssef Sarraf*
- *ASSOFI Nada*

Encadré par : Pr. Mohammed AIT DAOUD

Introduction générale :

La fraude bancaire représente un enjeu majeur pour le secteur financier : elle est rare, évolutive et noyée dans un volume massif de transactions légitimes, ce qui en fait un cas d'usage exigeant pour la Data Science appliquée. Le détecter efficacement ne se limite pas à entraîner un modèle de classification performant ; cela suppose de construire un **système** complet — ingestion, stockage, transformation, tests de qualité, orchestration, entraînement, suivi des expériences, mise en production et supervision — reproductible, traçable et testé à chaque étape.

Ce rapport présente le travail réalisé dans le cadre du Projet n°3 du catalogue du module MLOps & DataOps : la détection de transactions frauduleuses par classification (secteur Finance). Le projet applique concrètement les principes vus en cours — le cycle de vie DataOps en huit étapes (Chapitre 1), l'orchestration de pipelines agiles (Chapitre 2) et les tests de qualité des données (Chapitre 3) — à un cas réel : un jeu de données de 284 807 transactions dont seulement 0,17 % sont frauduleuses.

L'équipe a livré un pipeline DataOps complet (dlt → DuckDB → dbt → Dagster), un pipeline MLOps de bout en bout (préparation des données → entraînement → suivi MLflow → registre de modèles), un service de prédiction packagé (FastAPI + Docker), une chaîne d'intégration et de déploiement continu (CI/CD) et un dispositif de supervision (monitoring et détection de dérive). L'ensemble des 10 livrables attendus par le cahier des charges a été réalisé et vérifié.

Dans cette seconde version du rapport, le système a été étendu au-delà du périmètre initial : le jeu de données brut est désormais versionné avec **DVC** sur un stockage objet **MinIO** partagé ; une **plateforme d'entraînement continu** (platform-api) reçoit de nouvelles transactions, versionne les données lorsqu'un seuil est atteint, relance le pipeline complet et ne promeut le nouveau modèle en Production que s'il améliore le PR-AUC du modèle en place ; enfin, l'ensemble, désormais composé de six services (API de scoring, MLflow, Grafana, Prometheus, MinIO, platform-api), est déployé et redéployé automatiquement sur Komodo à chaque push.

La troisième version remplace le tableau de bord statique par une **plateforme d'opérations** destinée aux équipes Data et Risque : une application React authentifiée, servie par platform-api, qui permet de consulter et de corriger les transactions avec un versionnement DVC et un rollback, de lancer et de suivre le pipeline étape par étape, de surveiller la dérive des données et d'approuver ou de rejeter manuellement le modèle candidat. Elle est déployée sur Komodo avec le reste de la pile.

La quatrième version remplace l'instantané MLflow republié à chaque ré-entraînement par un serveur vivant (PostgreSQL, artefacts MinIO) partagé avec un vrai orchestrateur Dagster — schedule nocturne et sensor sur le dataset, jusque-là internes au conteneur d'entraînement continu. Elle ajoute des règles d'alerte Prometheus sur la dérive et l'état du pipeline, consultables dans un nouveau tableau de bord Grafana, et fait dépendre le déploiement Komodo du succès de la CI plutôt que d'un webhook inconditionnel.

Ce rapport est organisé en huit chapitres. Le premier présente la problématique et les objectifs du projet. Le deuxième détaille l'organisation Agile de l'équipe. Le troisième décrit l'architecture technique livrée, y compris le versioning des données (DVC, MinIO), la plateforme d'entraînement continu, l'interface d'opérations React et l'orchestration Dagster avec supervision de la dérive. Le quatrième synthétise les résultats clés, vérifiés et non estimés. Le cinquième établit la correspondance entre les 10 livrables attendus et leur réalisation. Le sixième rassemble les preuves d'exécution réelle des composants opérationnels (service de scoring, conteneurisation, CI/CD, supervision, déploiement sur Komodo, plateforme d'entraînement continu, interface d'opérations, orchestration Dagster et déploiement continu gardé par la CI). Le septième relate des exemples concrets d'ingénierie MLOps rencontrés durant le projet. Le huitième assume et documente les limites du travail ainsi que les perspectives d'amélioration.

Résumé :

Ce projet vise à détecter les transactions frauduleuses au sein d'un flux massif de transactions bancaires légitimes (0,17 % de fraude dans le jeu de données de référence), en construisant un système data/ML complet plutôt qu'un simple modèle. Le pipeline DataOps ingère les données via dlt, les stocke dans DuckDB, les transforme avec dbt (27 tests qualité, tous passés) et les orchestre avec Dagster. Le pipeline MLOps entraîne plusieurs modèles, retient un RandomForest à classes pondérées (PR-AUC de 0,8444 contre 0,7200 pour la baseline, rappel de 0,8163 et précision de 0,8421 au seuil optimal), et le trace avec MLflow (tracking et registry).

Le modèle est exposé via une API FastAPI conteneurisée avec Docker, validée par 12 tests unitaires à l'origine et testée en local comme en conteneur. Une chaîne CI/CD (lint → tests → ingestion → qualité → dbt → ML → registry → build Docker) a été validée dans un clone Git isolé, ce qui a permis de détecter et corriger un vrai bug de reproductibilité lié à la résolution de chemins par dlt. Un dispositif de monitoring calcule l'indice de stabilité de population (PSI) sur les variables clés, sans dérive significative observée sur la période couverte. Les 10 livrables du cahier des charges ont été réalisés et vérifiés, dans le cadre d'une organisation Agile en 3 sprints.

Dans sa seconde version, le projet ajoute le versioning du jeu de données avec DVC (remote MinIO partagé), une plateforme d'entraînement continu qui déclenche automatiquement versioning, ré-entraînement et enregistrement dans MLflow, et une porte de promotion qui n'élève un modèle en Production que si son PR-AUC dépasse celui du modèle actuel. La pile de huit services (API de scoring, MLflow, base PostgreSQL de MLflow, Grafana, Prometheus, MinIO, platform-api, Dagster) est déployée sur Komodo, avec un déploiement continu désormais gardé par la CI et des alertes de dérive et de pipeline ; la suite de tests compte désormais 71 tests automatisés, tous passants. Une interface d'opérations React authentifiée (transactions, versions du dataset avec rollback, pipeline, dérive, approbation du modèle) complète la plateforme.

Mots-clés : DataOps, MLOps, détection de fraude, déséquilibre de classes, dlt, DuckDB, dbt, Dagster, MLflow, FastAPI, Docker, CI/CD, monitoring, dérive de données, DVC, MinIO, Komodo, entraînement continu.

Abstract :

This project aims to detect fraudulent transactions within a massive stream of legitimate banking transactions (0.17% fraud in the reference dataset), by building a complete data/ML system rather than a single model. The DataOps pipeline ingests data via dlt, stores it in DuckDB, transforms it with dbt (27 quality tests, all passing) and orchestrates it with Dagster. The MLOps pipeline trains several models and retains a class-weighted RandomForest (PR-AUC of 0.8444 versus 0.7200 for the baseline, recall of 0.8163 and precision of 0.8421 at the optimal threshold), tracked end-to-end with MLflow (tracking and registry).

The model is served through a containerized FastAPI service, validated by 12 unit tests initially and exercised both locally and inside a Docker container. A CI/CD pipeline (lint → tests → ingestion → quality → dbt → ML → registry → Docker build) was validated in an isolated Git clone, which surfaced and fixed a genuine reproducibility bug tied to how dlt resolves relative paths. A monitoring component computes the Population Stability Index (PSI) on key features, with no significant drift observed over the period covered. All 10 deliverables required by the project specification were completed and verified, within a 3-sprint Agile organization.

In its second version, the project adds dataset versioning with DVC (shared MinIO remote), a continuous-training platform that automatically triggers versioning, retraining and MLflow registration, and a promotion gate that only promotes a model to Production if its PR-AUC beats the current one. The stack of eight services (scoring API, MLflow, MLflow's PostgreSQL database, Grafana, Prometheus, MinIO, platform-api, Dagster) is deployed on Komodo, with deployment now gated by CI and drift/pipeline alerting; the test suite now has 71 automated tests, all passing. An authenticated React operations interface (transactions, dataset versions with rollback, pipeline, drift, model approval) completes the platform.

Keywords: DataOps, MLOps, fraud detection, class imbalance, dlt, DuckDB, dbt, Dagster, MLflow, FastAPI, Docker, CI/CD, monitoring, data drift, DVC, MinIO, Komodo, continuous training.

Table des matières :

Chapitre 1 : Problématique et objectifs du projet.....	13
1. Contexte et problématique :.....	13
2. Objectifs du projet :.....	14
Chapitre 2 : Organisation Agile du projet.....	14
1. Équipe et rôles :.....	14
2. Sprints réalisés :.....	15
3. Product Backlog :.....	17
Chapitre 3 : Architecture technique livrée.....	17
1. Vue d'ensemble :.....	17
2. Composants technologiques :.....	18
3. Versioning des données : DVC et MinIO :.....	22
4. Plateforme d'entraînement continu :.....	24
5. Interface d'opérations : plateforme bancaire (React) :.....	27
6. Orchestration Dagster, MLflow serveur et supervision :.....	27
Chapitre 4 : Résultats clés.....	28
1. Indicateurs vérifiés :.....	28
2. Interprétation :.....	32
Chapitre 5 : Mapping des 10 livrables du cahier des charges.....	34
Chapitre 6 : Démonstration opérationnelle des composants livrés.....	36
1. Service de scoring (FastAPI) :.....	36
2. Conteneurisation (Docker) :.....	37
3. Intégration et déploiement continus (CI/CD) :.....	38
4. Supervision et détection de dérive :.....	39
5. Tests automatisés :.....	41
6. Déploiement continu sur Komodo :.....	42
7. Plateforme d'entraînement continu en fonctionnement :.....	45
8. Interface d'opérations en fonctionnement :.....	46
9. Orchestration Dagster, MLflow serveur et supervision de la dérive :.....	55
10. Déploiement continu gardé par la CI :.....	57
Chapitre 7 : Exemples concrets d'ingénierie MLOps rencontrés pendant le projet.....	58
Chapitre 8 : Limites assumées et perspectives.....	60

Liste des figures :

Figure 1 : Déséquilibre des classes du jeu de données de référence — 0,17 % de transactions frauduleuses parmi 284 807 transactions.

Figure 2 : Architecture technique livrée — pipeline DataOps et MLOps de bout en bout.

Figure 3 : PR-AUC du modèle retenu contre la baseline, et rappel/précision au seuil optimal.

Figure 4 : L'équipe projet au complet (8 membres, rôles Agile détaillés au Tableau 1).

Figure 5 : Sprint 1 — Planning / Review / Rétrospective (fondations DataOps : dlt, DuckDB, dbt, Dagster).

Figure 6 : Sprint 2 — Planning / Review / Rétrospective (qualité des données et Machine Learning / MLflow).

Figure 7 : Sprint 3 — Planning / Review / Rétrospective (déploiement, CI/CD, monitoring).

Figure 8 : Terminal après exécution du pipeline dlt — résumé de chargement (284 807 lignes).

Figure 9 : Requête DuckDB sur stg_transactions confirmant 492 fraudes sur 284 807 transactions.

Figure 10 : Vue graphe Dagit du job fraud_pipeline_job, tous les nœuds en succès.

Figure 11 : Résumé dbt test — 27/27 tests PASS.

Figure 12 : Graphe de lignage dbt (Data Lineage).

Figure 13 : Validation shift-left du schéma — sortie terminal.

Figure 14 : Extrait du Data Contract (docs/data/data_contract.yaml).

Figure 15 : Comparaison des runs MLflow — PR-AUC 0,7200 (baseline) vs 0,8444 (retenu).

Figure 16 : Détail du run rf_balanced_v1 (paramètres, métriques, artefacts).

Figure 17 : Model Registry MLflow — fraud-detection-classifier au stage Production.

Figure 18 : Courbe Précision-Rappel — modèle retenu (rf_balanced_v1), PR-AUC = 0,8444.

Figure 19 : Matrice de confusion — modèle retenu (recall 0,8163 / precision 0,8421).

Figure 20 : Courbe Précision-Rappel — baseline (logreg_baseline), PR-AUC = 0,7200.

Figure 21 : Matrice de confusion — baseline (logreg_baseline).

Figure 22 : Documentation Swagger du service de scoring — endpoints /health, /predict, /metrics.

Figure 23 : Appels curl réels sur /health et /predict, avec la réponse JSON (fraud_probability, is_fraud, model_version).

Figure 24 : Fin du build Docker (image fraud-api taguée avec succès).

Figure 25 : Appel curl /health exécuté contre le conteneur Docker, confirmant la portabilité du service.

Figure 26 : Run GitHub Actions complet, toutes les étapes passées avec succès.

Figure 27 : Sortie Prometheus de l'endpoint /metrics du service en fonctionnement.

Figure 28 : Sortie terminal du contrôle de dérive (PSI par variable).

Figure 29 : Extrait du rapport de dérive généré (monitoring/drift_report.md).

Figure 30 : Résultat final de la suite de tests pytest — 12 passed (suite initiale).

Figure 31 : Interface Komodo — les six services du stack à l'état RUNNING sur le serveur vh3.

Figure 32 : Versioning du jeu de données — Git ne conserve que le pointeur .dvc ; le contenu est stocké dans MinIO.

Figure 33 : Boucle d'entraînement continu — seuil, versioning DVC, job Dagster et porte de promotion.

Figure 34 : Historique des runs de la plateforme d'opérations (instance Komodo).

Figure 35 : Architecture de déploiement sur Komodo — six services sur le réseau fraud_net.

Figure 36 : Cible Prometheus fraud-api à l'état UP sur Komodo.

Figure 37 : Console MinIO — deux versions du dataset dans le bucket dvc-store.

Figure 38 : Tableau de bord Grafana « Fraud Detection API » sur Komodo.

Figure 39 : Page de connexion de la plateforme d'opérations (instance Komodo).

Figure 40 : Refus d'un mot de passe erroné par l'API (instance locale de démonstration).

Figure 41 : Transactions : filtre « Fraude » (492 lignes) et composantes V1–V28 d'une ligne dépliée.

Figure 42 : Tiroir d'édition d'une transaction.

Figure 43 : Correction en attente : ligne surlignée, pastille dans le menu et barre d'enregistrement.

Figure 44 : Confirmation « Enregistrer et versionner » (dvc add + commit git).

Figure 45 : Job du pipeline en cours d'exécution, suivi étape par étape.

Figure 46 : Job terminé et comparaison Production / candidat (Staging).

Figure 47 : Page Dérive après simulation : alerte PSI 0,489, évolution et distribution.

Figure 48 : Page Dérive de l'instance Komodo : PSI 0,003 (stable).

Figure 49 : Confirmation d'approbation du candidat : métriques avant / après.

Figure 50 : Confirmation de rollback du dataset.

Figure 51 : Versions du dataset après rollback : nouveau commit « Rollback... ».

Figure 52 : Historique des runs : run de relance déplié avec étapes et journal.

Figure 53 : Page Écosystème de l'instance Komodo (22 septembre 2026) : MLflow, Grafana, Prometheus, API de scoring, Dagster, MinIO et API de la plateforme — sept services, tous « disponibles ».

Figure 54 : Automation de l'UI Dagster : sensor `dataset\_version\_sensor` et schedule `nightly\_pipeline\_schedule` (02:00).

Figure 55 : Run Dagster `continuous\_training\_job` déclenché depuis le tableau de bord : statut Success, 4 min 26 s.

Figure 56 : Modèle `fraud-detection-classifier` version 1, registre neuf : « Stage (deprecated) : Staging ».

Figure 57 : Tableau de bord Grafana « Dérive & pipeline » de l'instance Komodo : PSI 0,003, dernier run SUCCÈS.

Figure 58 : Interface Prometheus /alerts : sept règles de dérive, de pipeline et de disponibilité, toutes inactives.

Figure 59 : Exécution GitHub Actions gardée par la CI : job `test` puis job « Déploiement Komodo (si les tests passent) ».

Figure 60 : Page Pipeline & automatisation : promotion automatique désactivée et lien « Ouvrir l'UI Dagster ».

Liste des tableaux :

Tableau 1 : Composition de l'équipe et répartition des rôles Agile.

Tableau 2 : Composants technologiques du pipeline et leur rôle.

Tableau 3 : Indicateurs clés vérifiés du projet.

Tableau 4 : Mapping des 10 livrables du cahier des charges.

Tableau 5 : Points d'entrée de la plateforme d'opérations et d'entraînement continu.

Tableau 6 : Services du stack déployé sur Komodo.

Table des abréviations :

Abréviation	Signification
ML	Machine Learning (apprentissage automatique)
DataOps	Data Operations (opérationnalisation des données)
MLOps	Machine Learning Operations (opérationnalisation du ML)
dbt	data build tool
ETL / ELT	Extract-Transform-Load / Extract-Load-Transform
CI/CD	Continuous Integration / Continuous Deployment (intégration et déploiement continus)
API	Application Programming Interface (interface de programmation)
REST	Representational State Transfer
PR-AUC	Precision-Recall Area Under Curve (aire sous la courbe précision-rappel)
PSI	Population Stability Index (indice de stabilité de population)
PO	Product Owner
SM	Scrum Master
JSON	JavaScript Object Notation
YAML	YAML Ain't Markup Language
DVC	Data Version Control (versioning de données)
S3	Simple Storage Service (protocole de stockage objet)

SPA	Single-Page Application (application web monopage)
HMAC	Hash-based Message Authentication Code (signature du jeton)
CRUD	Create, Read, Update, Delete (création, lecture, modification, suppression)

Chapitre 1 : Problématique et objectifs du projet

1. Contexte et problématique :

La détection de fraude bancaire consiste à identifier des transactions frauduleuses parmi un flux massif de transactions légitimes. Dans le jeu de données de référence utilisé pour ce projet, sur 284 807 transactions, seules 492 sont frauduleuses, soit 0,17 % — un déséquilibre extrême qui rend le problème difficile à la fois statistiquement (un modèle naïf qui prédit toujours « légitime » atteint 99,83 % d'exactitude sans détecter la moindre fraude) et opérationnellement (le coût d'un faux négatif — une fraude non détectée — n'est pas le même que celui d'un faux positif — un client légitime bloqué).

Le détail complet de la problématique, des utilisateurs cibles, de la valeur métier et de la Data Strategy est disponible dans docs/01_vision.md.

Déséquilibre des classes du dataset de référence

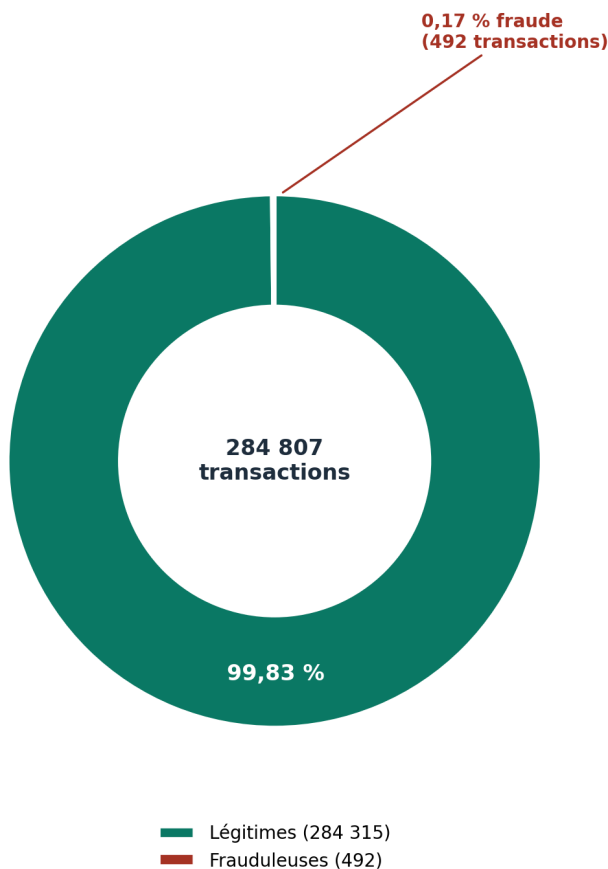


Figure 1 : Déséquilibre des classes du jeu de données de référence — 0,17 % de transactions frauduleuses parmi 284 807 transactions.

2. Objectifs du projet :

L'objectif n'est pas seulement de produire un modèle de classification, mais de construire un **système** data/ML complet, reproductible, traçable, testé et déployé, conformément au cadre DataOps/MLOps vu en cours (Atwal 2019, DataOps Manifesto, cycle de vie en 8 étapes du Chapitre 1). Concrètement, le projet vise à :

- Mettre en place un pipeline d'ingestion et de transformation des données automatisé et versionné (dlt, DuckDB, dbt).
- Versionner le jeu de données brut avec DVC, sur un stockage objet MinIO partagé, afin de retrouver exactement les données ayant servi à chaque entraînement.
- Garantir la qualité des données par des tests systématiques adossés à un Data Contract.
- Orchestrer l'ensemble du pipeline de données avec Dagster.
- Entraîner, évaluer et comparer plusieurs modèles de classification adaptés au déséquilibre de classes.
- Tracer les expériences et gérer le cycle de vie du modèle avec MLflow (tracking et registry).
- Déployer le modèle retenu comme service exposé via une API, conteneurisé avec Docker.
- Déployer l'ensemble des services sur le serveur Komodo mis à disposition, avec redéploiement automatique à chaque push Git.
- Automatiser l'intégration et le déploiement continu (CI/CD).
- Automatiser le ré-entraînement (plateforme d'entraînement continu) avec une porte de promotion qui empêche un modèle moins performant d'atteindre la Production.
- Mettre en place un dispositif de supervision et de détection de dérive en production.

Chapitre 2 : Organisation Agile du projet

1. Équipe et rôles :

Le projet a été mené par une équipe de huit personnes, avec une répartition des rôles Agile résumée dans le Tableau 1 (cumul de rôles possible, conformément à la taille de l'équipe).

Tableau 1 : Composition de l'équipe et répartition des rôles Agile.

Rôle	Membre(s)
Data Engineer, Product Owner	Hassan El Hadi
Data Engineer, Scrum Master	Yassin Farih

Data Engineer (lead)	Anass Dabibe
Data Engineer	Adnane Dahbi
Data Engineer	Sifeddine Legnioui
ML Engineer	Aymane El Badri
ML Engineer	Youssef Sarraf
Data Analyst	ASSOFI Nada


MLOPS-TEAM

 Équipe officielle

[À propos](#)
[Activité](#)
[Hiérarchie](#)
[Objectifs](#)
[Projets](#)
[Félicitations](#)

Ce que nous faisons

Indiquez ce que fait votre équipe

Membres 8


 anass dabibe
Web Developer


 adnane dahbi


 Assofi Nada


 Aymane El badry
Web Developer


 Hassan El Hadi


 SIFEDDINE LEGNIOUI


 YASSINE FARIH


 Youssef Sarraf

Figure 4 : L'équipe projet au complet (8 membres, rôles Agile détaillés au Tableau 1).

2. Sprints réalisés :

Conformément au minimum imposé par le cahier des charges, l'équipe a réalisé trois sprints, chacun structuré autour d'une Planning, d'une Review et d'une Rétrospective (détail dans docs/agile/) :

- **Sprint 1** : fondations du projet et pipeline DataOps (dlt / DuckDB / dbt / Dagster).

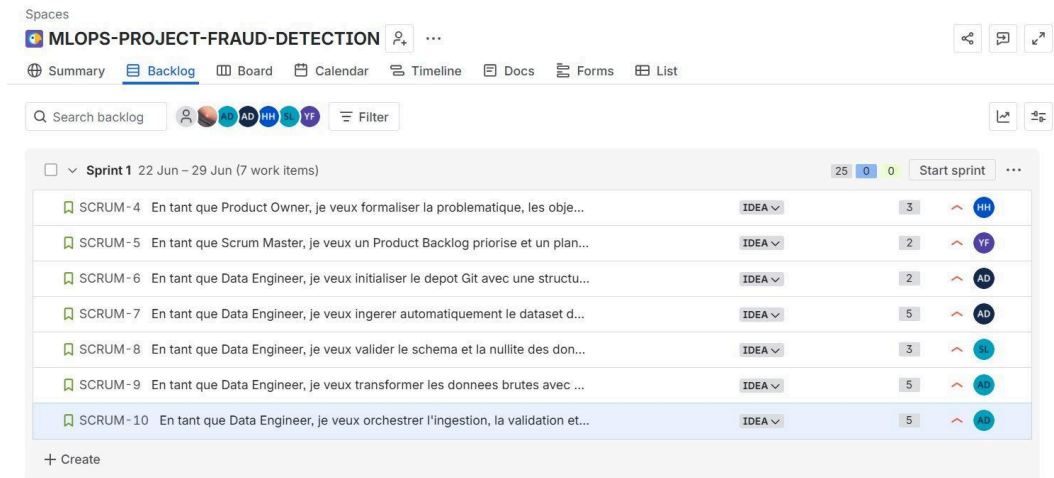


Figure 5 : Sprint 1 — Planning / Review / Rétrospective (fondations DataOps : dlt, DuckDB, dbt, Dagster).

- **Sprint 2** : qualité des données (Data Contract, lignage) et Machine Learning / MLflow.

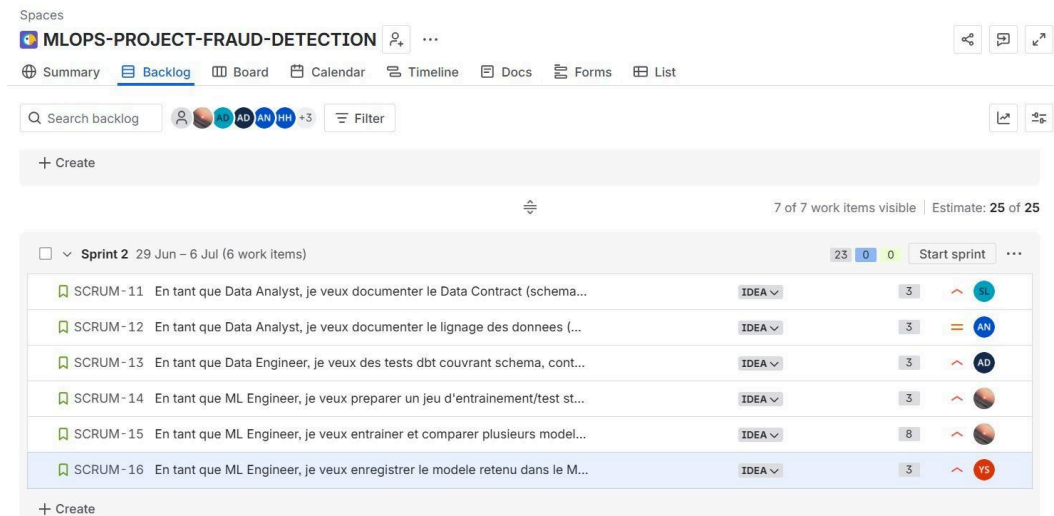


Figure 6 : Sprint 2 — Planning / Review / Rétrospective (qualité des données et Machine Learning / MLflow).

- **Sprint 3** : déploiement (FastAPI / Docker), CI/CD, monitoring.

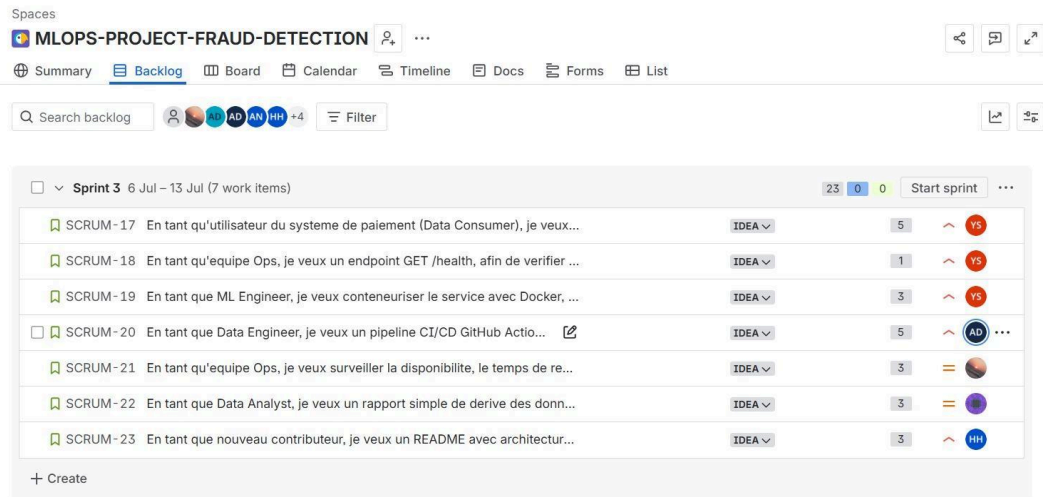


Figure 7 : Sprint 3 — Planning / Review / Rétrospective (déploiement, CI/CD, monitoring).

3. Product Backlog :

Le Product Backlog comprend 20 user stories couvrant les 10 livrables attendus par le cahier des charges (docs/agile/product_backlog.md).

Chapitre 3 : Architecture technique livrée

1. Vue d'ensemble :

L'architecture livrée enchaîne un pipeline DataOps (ingestion, stockage, transformation, qualité, orchestration) et un pipeline MLOps (entraînement, tracking, déploiement, supervision), comme illustré en Figure 2. Le détail complet est disponible dans docs/02_architecture.md, et le lignage bout en bout dans docs/data/data_lineage.md.

L'ingestion et le stockage ne sont pas de simples affirmations : l'exécution du pipeline dlt (Figure 8) confirme le chargement effectif des 284 807 transactions dans l'entrepôt DuckDB, et une requête directe sur la table stg_transactions (Figure 9) retrouve exactement les 492 fraudes (0,17 %) citées au Chapitre 1 et au Tableau 3 — validant que le déséquilibre de classes annoncé n'est pas une estimation mais une donnée vérifiable en base.

```
(python_3_11_13) (.venv) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$
python ingestion_dlt/run_pipeline.py

Pipeline fraud_ingestion load step completed in 1 minute and 3.54 seconds
1 load package(s) were loaded to destination duckdb and into dataset raw
The duckdb destination used duckdb:///home/kappa/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops/fr
aud_detection.duckdb location to store data
Load package 1783683233.2208047 is LOADED and contains no failed jobs
```

Figure 8 : Terminal après exécution du pipeline dlt — résumé de chargement (284 807 lignes).

```
(python_3_11_13) (.venv) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$
• python -c "
import duckdb
con = duckdb.connect('fraud detection.duckdb', read_only=True)
print(con.execute('SELECT is_fraud, COUNT(*) FROM main.stg_transactions GROUP BY is_fraud').fetchdf())
"
  is_fraud  count_star()
0         0        284315
1         1         492
```

Figure 9 : Requête DuckDB sur stg_transactions confirmant 492 fraudes sur 284 807 transactions.

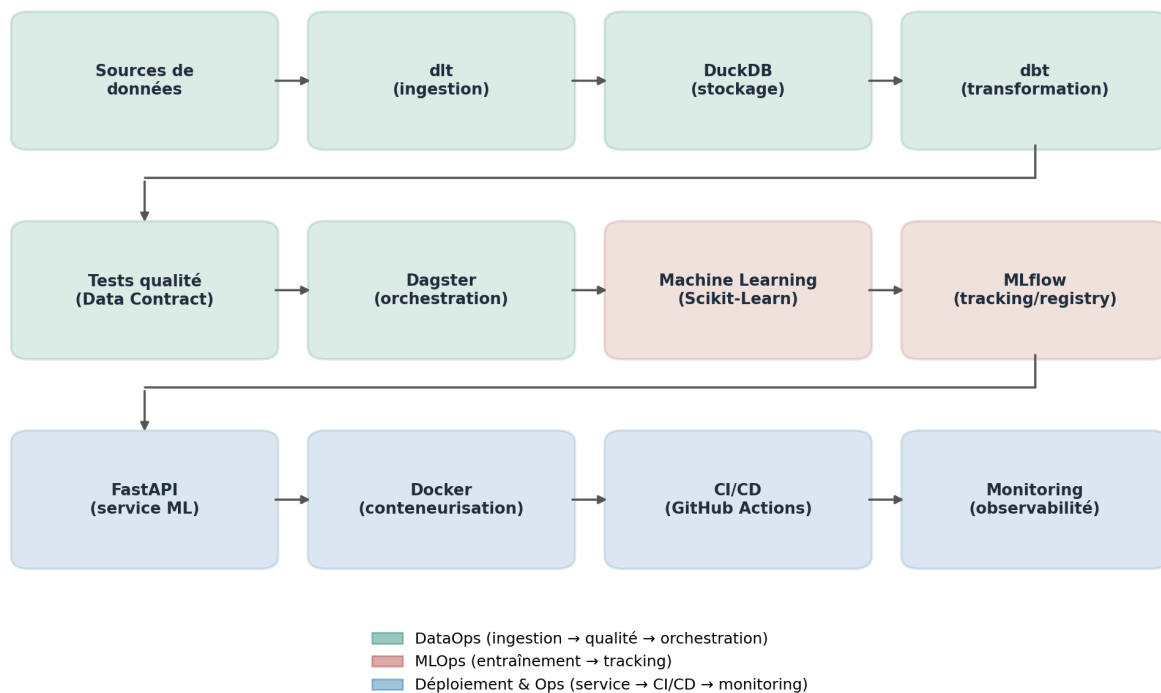


Figure 2 : Architecture technique livrée — pipeline DataOps et MLOps de bout en bout.

2. Composants technologiques :

Tableau 2 : Composants technologiques du pipeline et leur rôle.

Composant	Rôle	Technologie
Ingestion	Automatiser la collecte des transactions vers l'entrepôt	dlt
Stockage	Stocker les données ingérées localement	DuckDB

Transformation	Modéliser et transformer les données, exécuter les tests qualité	dbt
Orchestration	Planifier et exécuter le pipeline de données (fraud_pipeline_job) et le pipeline d'entraînement continu (continuous_training_job), partagés avec le tableau de bord via un schedule nocturne et un sensor sur le dataset	Dagster
Modélisation	Entraîner et évaluer les modèles de classification	Scikit-Learn
Tracking & Registry	Tracer les expériences, gérer le cycle de vie du modèle et n'autoriser la promotion en Production que sous condition de PR-AUC ; servi par un serveur vivant (PostgreSQL, artefacts MinIO)	MLflow
Service ML	Exposer le modèle retenu via une API	FastAPI
Conteneurisation	Packager le service pour un déploiement reproductible	Docker
CI/CD	Automatiser lint, tests, déploiement (gardé par le succès des tests, avec retries) à chaque changement	GitHub Actions

Monitoring	Superviser le service, détecter la dérive des données et alerter (Prometheus, Grafana) en cas de dérive ou d'échec du pipeline	Prometheus (scrape GET /metrics), Grafana (dashboards), scripts PSI
Déploiement	Construire et exécuter automatiquement les huit services sur un serveur distant à partir du dépôt Git (redéploiement par webhook)	Komodo
Versioning des données	Suivre le jeu de données brut par un pointeur versionné dans Git, sans stocker le fichier dans Git	DVC
Stockage objet	Héberger le remote DVC partagé (bucket dvc-store), compatible S3	MinIO
Entraînement continu	Recevoir de nouvelles données, versionner, relancer le pipeline et contrôler la promotion du modèle	FastAPI (platform-api)

Les trois dernières lignes du Tableau 2 (DVC, MinIO, entraînement continu) sont détaillées aux sections 3 et 4 de ce chapitre. Chaque autre ligne du Tableau 2 est illustrée ci-dessous par une exécution réelle plutôt que par une simple description. L'orchestration Dagster (Figure 10) exécute l'ensemble du graphe — ingestion, validation, dbt run, dbt test — avec tous les nœuds passés avec succès. Les tests de qualité dbt (Figure 11) confirment le chiffre de 27/27 PASS déjà cité au Tableau 3, et le graphe de lignage généré par dbt docs (Figure 12) matérialise concrètement le Data Lineage exigé par le Livrable 4. Enfin, la validation shift-left du schéma (Figure 13) et l'extrait du Data Contract (Figure 14) relie le concept théorique de contrat de données à un fichier réel (docs/data/data_contract.yaml) et à une vérification exécutée avant même le chargement en base.

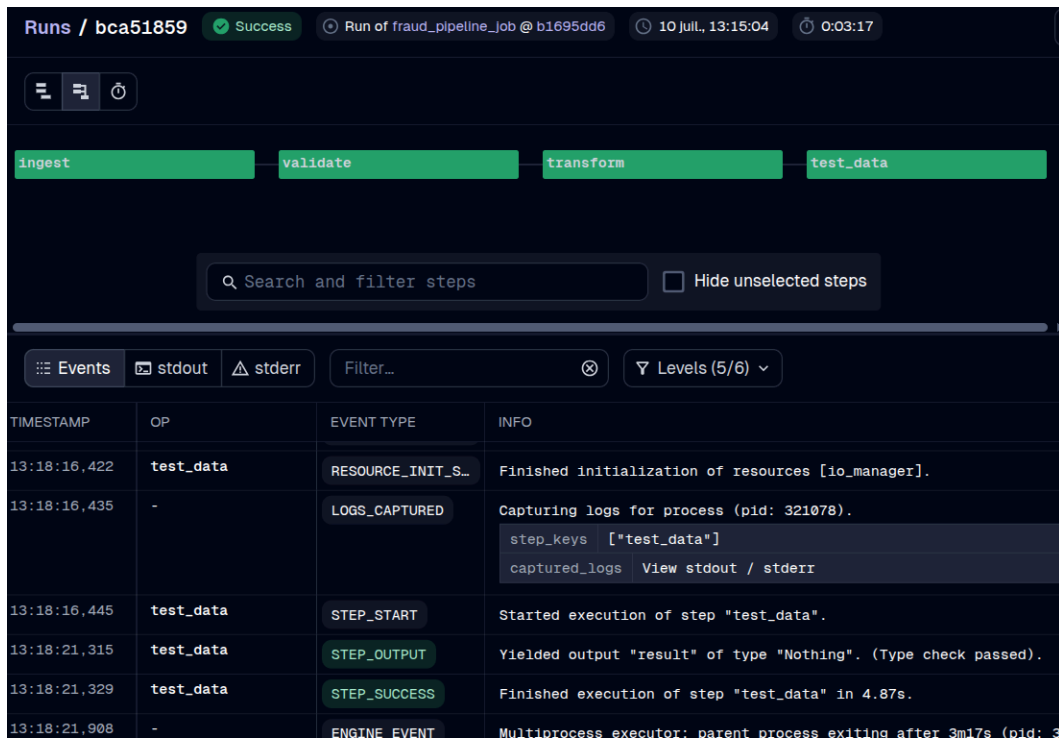


Figure 10 : Vue graphe Dagit du job fraud_pipeline_job, tous les nœuds en succès.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(python 3 11 13) (.venv) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataapps/fraud-detection-mlops/dbt_fraud
$ dbt test --profiles-dir .
12:03:48 3 of 23 PASS accepted values fct transactions clean is fraud 0 1 ..... [PASS in 0.03s]
12:03:48 4 of 23 START test accepted values fct transactions features amount bucket zero low medium high ver
y high [RUN]
12:03:48 4 of 23 PASS accepted values fct transactions features amount bucket zero low medium high very high
[PASS in 0.05s]
12:03:48 5 of 23 START test accepted values stg transactions is fraud 0 1 ..... [RUN]
12:03:48 5 of 23 PASS accepted values stg transactions is fraud 0 1 ..... [PASS in 0.02s]
12:03:48 6 of 23 START test assert amount non negative ..... [RUN]
12:03:48 6 of 23 PASS assert amount non negative ..... [PASS in 0.02s]
12:03:48 7 of 23 START test not null agg fraud hourly summary fraud rate ..... [RUN]
12:03:48 7 of 23 PASS not null agg fraud hourly summary fraud rate ..... [PASS in 0.02s]
12:03:48 8 of 23 START test not null agg fraud hourly summary hour of day ..... [RUN]
12:03:48 8 of 23 PASS not null agg fraud hourly summary hour of day ..... [PASS in 0.02s]
12:03:48 9 of 23 START test not null fct transactions clean amount ..... [RUN]
12:03:48 9 of 23 PASS not null fct transactions clean amount ..... [PASS in 0.03s]
12:03:48 10 of 23 START test not null fct transactions clean is fraud ..... [RUN]
12:03:48 10 of 23 PASS not null fct transactions clean is fraud ..... [PASS in 0.03s]
12:03:48 11 of 23 START test not null fct transactions clean transaction id ..... [RUN]
12:03:48 11 of 23 PASS not null fct transactions clean transaction id ..... [PASS in 0.02s]
12:03:48 12 of 23 START test not null fct transactions features amount bucket ..... [RUN]
12:03:48 12 of 23 PASS not null fct transactions features amount bucket ..... [PASS in 0.02s]
12:03:48 13 of 23 START test not null fct transactions features hour of day ..... [RUN]
12:03:48 13 of 23 PASS not null fct transactions features hour of day ..... [PASS in 0.02s]
12:03:48 14 of 23 START test not null fct transactions features log amount ..... [RUN]
12:03:48 14 of 23 PASS not null fct transactions features log amount ..... [PASS in 0.02s]
12:03:48 15 of 23 START test not null fct transactions features transaction id ..... [RUN]
12:03:48 15 of 23 PASS not null fct transactions features transaction id ..... [PASS in 0.02s]
12:03:48 16 of 23 START test not null stg transactions amount ..... [RUN]
12:03:48 16 of 23 PASS not null stg transactions amount ..... [PASS in 0.02s]
12:03:48 17 of 23 START test not null stg transactions is fraud ..... [RUN]
12:03:48 17 of 23 PASS not null stg transactions is fraud ..... [PASS in 0.02s]
12:03:48 18 of 23 START test not null stg transactions time seconds ..... [RUN]
12:03:48 18 of 23 PASS not null stg transactions time seconds ..... [PASS in 0.02s]
12:03:48 19 of 23 START test not null stg transactions transaction id ..... [RUN]
12:03:48 19 of 23 PASS not null stg transactions transaction id ..... [PASS in 0.02s]
12:03:48 20 of 23 START test unique agg fraud hourly summary hour of day ..... [RUN]
12:03:48 20 of 23 PASS unique agg fraud hourly summary hour of day ..... [PASS in 0.02s]
12:03:48 21 of 23 START test unique fct transactions clean transaction id ..... [RUN]
12:03:48 21 of 23 PASS unique fct transactions clean transaction id ..... [PASS in 0.04s]
12:03:48 22 of 23 START test unique fct transactions features transaction id ..... [RUN]
12:03:48 22 of 23 PASS unique fct transactions features transaction id ..... [PASS in 0.04s]
12:03:48 23 of 23 START test unique stg transactions transaction id ..... [RUN]
12:03:48 23 of 23 PASS unique stg transactions transaction id ..... [PASS in 0.04s]
12:03:48
12:03:48 Finished running 23 data tests in 0 hours 0 minutes and 0.81 seconds (0.81s).
12:03:48
12:03:48 Completed successfully
12:03:48
12:03:48 Done. PASS=23 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=23

```

Figure 11 : Résumé dbt test — 27/27 tests PASS.

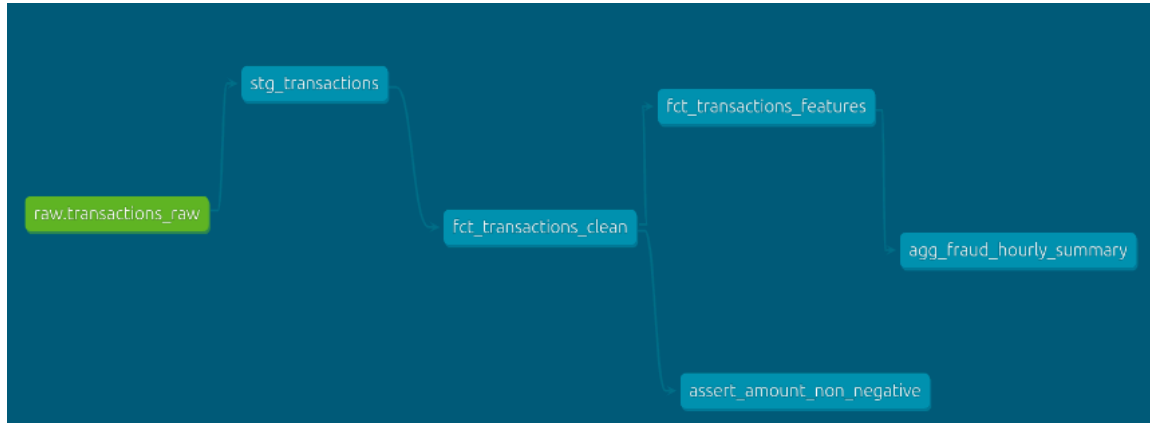


Figure 12 : Graphe de lignage dbt (Data Lineage).

```

cd fraud-detection-mlops/
(pyhton_3_11_13) (.venv) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$
• python quality/validate_schema.py
Validation réussie : 284807 lignes, schéma et qualité minimale OK
  
```

Figure 13 : Validation shift-left du schéma — sortie terminal.

```

# --- Règles de qualité (mappées sur les 11 dimensions du Chapitre 3) ---
quality_checks:
  - dimension: completeness
    rule: "aucune valeur nulle sur transaction_id, amount, is_fraud, time_seconds"
    implementation: dbt not_null (models/staging/stg_transactions.yml, models/marts/marts.yml)
  - dimension: validity
    rule: "is_fraud in (0,1) ; amount >= 0 ; hour_of_day in [0,23] ; amount_bucket in valeurs autorisées"
    implementation: dbt accepted_values + test générique accepted_range (macros/generic_tests.sql)
  - dimension: uniqueness
    rule: "transaction_id unique sur l'ensemble des tables du contrat"
    implementation: dbt unique
  - dimension: business_consistency
    rule: "amount doit être positif ou nul (aucune transaction à montant négatif)"
    implementation: dbt_fraud/tests/assert_amount_non_negative.sql (test singulier)
  - dimension: traceability
    rule: "chaque ligne porte le load_id du run dlt qui l'a produite"
    implementation: colonne load_id propagée depuis raw.transactions_raw jusqu'aux marts
  - dimension: documentation
    rule: "chaque colonne exposée est décrite dans les fichiers schema.yml dbt"
    implementation: dbt_fraud/models/**/*.yml
  - dimension: freshness
    rule: "non applicable au sens SLA temps réel - dataset historique statique"
    implementation: "voir docs/data/data_quality_report.md (limite assumée et documentée)"
  - dimension: integrity
    rule: "non applicable - schéma en étoile à une seule table de faits, pas de relation référentielle à contrôler"
    implementation: "voir docs/data/data_quality_report.md"
  
```

Figure 14 : Extrait du Data Contract (docs/data/data_contract.yaml).

3. Versioning des données : DVC et MinIO :

Le jeu de données brut (data/raw/creditcard.csv, 150 828 752 octets, soit environ 144 Mo) n'est pas stocké dans Git : il dépasse ce qu'un dépôt de code doit contenir et sa licence (Kaggle ODbL) restreint sa redistribution. Il est suivi par **DVC** (Data Version Control) : Git ne versionne qu'un petit fichier pointeur,

data/raw/creditcard.csv.dvc, qui contient l'empreinte MD5 et la taille du fichier réel (Figure 32) ; le contenu lui-même est envoyé vers un stockage distant (*remote*).

Le remote est un service **MinIO** (stockage objet compatible S3) ajouté à docker-compose.yml, avec un bucket dvc-store (API sur le port 4605, console web sur le port 4606). L'image est tirée depuis quay.io/minio/minio, épinglée sur la version RELEASE.2025-04-22T22-12-26Z, car les images MinIO du Docker Hub exigent désormais une connexion. Les identifiants ne sont jamais commités : .dvc/config.local est ignoré par Git, et .dvc/config.local.example documente l'adresse à utiliser selon le contexte (poste local http://localhost:4605, poste d'un coéquipier vers Komodo http://exp.s3.fsbm.ma:4605, conteneur à conteneur http://minio:9000).

Ce dispositif remplace le téléchargement manuel depuis Kaggle par un simple dvc pull, et rend chaque version du jeu de données traçable : une nouvelle version correspond à un nouveau commit du fichier .dvc, donc à un état précis du code et des données que l'on peut retrouver ensemble. Le contenu réel des deux fichiers de configuration versionnés est reproduit ci-dessous.

Contenu réel de data/raw/creditcard.csv.dvc (pointeur) et de .dvc/config (déclaration du remote) :

```
# data/raw/creditcard.csv.dvc
outs:
- md5: e90efcb83d69faf99fcab8b0255024de
  size: 150828752
  hash: md5
  path: creditcard.csv

# .dvc/config
[core]
  analytics = false
  remote = minio_shared
['remote "minio_shared"']
  url = s3://dvc-store
```

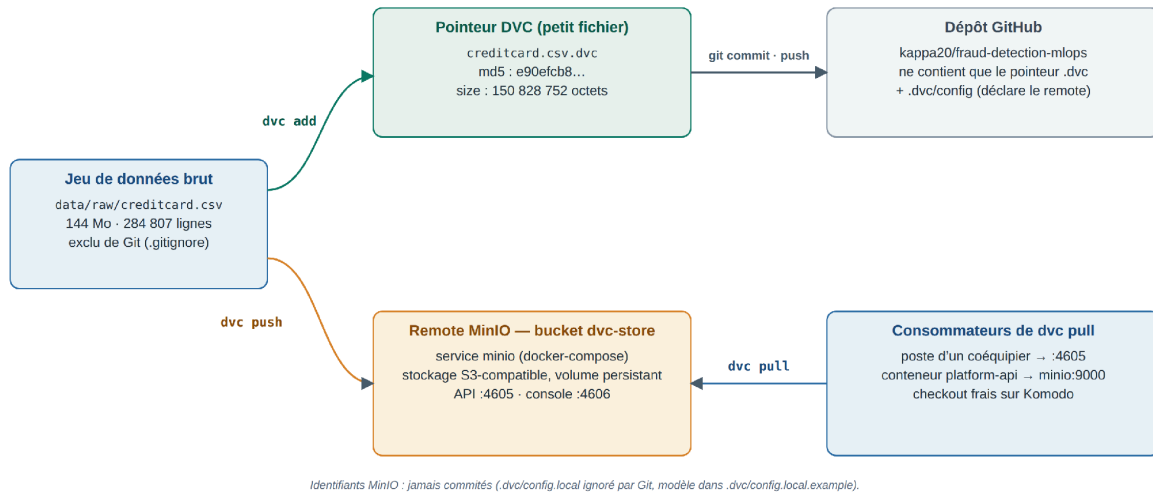


Figure 32 : Versioning du jeu de données — Git ne conserve que le pointeur .dvc ; le contenu est stocké dans le bucket MinIO dvc-store.

4. Plateforme d'entraînement continu :

Pour simuler le fonctionnement d'une banque, qui reçoit en continu de nouvelles transactions dont le caractère frauduleux n'est confirmé qu'après coup, le projet comprend un second service FastAPI, **platform-api** (dossier platform_api/, image Dockerfile.platform), distinct de l'API de scoring temps réel. Il expose les points d'entrée du Tableau 5 et l'interface d'opérations décrite au §5 (Figures 34 et 39 à 53, Chapitre 6).

Tableau 5 : Points d'entrée de la plateforme d'opérations et d'entraînement continu.

Endpoint	Rôle
POST /auth/login	Authentification : renvoie un jeton signé (HMAC, validité 8 h).
GET /health	Contrôle de disponibilité (sans authentification).
GET /	Interface React (fichiers statiques sous /assets).
GET /status, GET / PUT /config	État du déclencheur ; seuil (100 par défaut), seuil PSI, ré-entraînement automatique, promotion automatique.
POST /data/ingest, /data/simulate	Ajouter des transactions étiquetées ; générer $n \leq 5\,000$ transactions synthétiques avec une intensité de dérive (none, moderate, strong).

GET /transactions, /transactions/{id}, /transactions/export	Table paginée côté serveur (tri, filtres), détail d'une ligne, export CSV de la sélection.
POST /transactions/changes	Créations, modifications et suppressions par lot : réécriture atomique du CSV, dvc add + push, commit git (bot) ; annulation si le versionnement échoue ; HTTP 409 si le lot a été préparé sur une version périmée.
GET /versions, POST /versions/{sha}/rollback	Historique des versions du dataset ; restauration sous la forme d'un nouveau commit.
POST /pipeline/run, /pipeline/retrain	Pipeline complet ou ré-entraînement (Dagster) ; un seul job à la fois.
GET /jobs/{id}, GET /runs, POST /runs/{id}/rerun	Suivi d'un job étape par étape, historique des runs, relance.
GET /drift/current, /drift/history, /drift/distribution, POST /drift/check	PSI courant par variable, historique, distributions ; recalcul et journalisation.
GET /models/comparison, POST /models/approve, /models/reject	Production contre candidat ; approbation (promotion et redémarrage) ou rejet.
POST /models/redeploy	Redémarrage du service de scoring par l'API Komodo — à valider sur Komodo, non testé hors Komodo.
GET /services/health	Disponibilité des sept services de l'écosystème (vérifications depuis le serveur).

Mécanisme de déclenchement. Chaque ingestion ajoute les lignes à data/raw/creditcard.csv et incrémente un compteur de mises à jour en attente. Ce compteur et le seuil sont persistés dans data/state/pipeline_state.json plutôt qu'en mémoire : un redéploiement du conteneur, fréquent sur Komodo, ne doit ni remettre le compteur à zéro ni perdre le seuil configuré. Lorsque le seuil est atteint, la plateforme (1) versionne le dataset (dvc add, dvc push, puis git commit local) et (2) lance en tâche de fond le job Dagster continuous_training_job : ingestion, validation, dbt run, dbt test, entraînement, contrôle de dérive, enregistrement et promotion (Figure 33). L'exécution en tâche de fond (BackgroundTasks de FastAPI) permet à l'appel d'ingestion de répondre immédiatement, alors que le ré-entraînement d'un RandomForest sur environ 285 000 lignes prend plusieurs dizaines de secondes.

Simulation de dérive. Des lignes ré-échantillonnées sans perturbation auraient une distribution quasi identique à la référence et le PSI resterait proche de 0. Le paramètre `drift_intensity` applique donc des perturbations délibérées : *moderate* (montants multipliés par 1,8, taux de fraude $\times 3$, bruit gaussien d'écart-type 1,0 sur les variables V1–V28) et *strong* (montants $\times 3,5$, taux de fraude $\times 8$, bruit d'écart-type 2,5). Les lignes sont toujours tirées des 5 000 premières lignes du dataset d'origine, afin que la référence de comparaison ne dérive pas avec les ajouts de la plateforme.

Porte de promotion. Sans garde-fou, un ré-entraînement sur des données bruitées ou dérivées pourrait dégrader silencieusement le modèle en production à chaque déclenchement automatique. Le script `ml/register_model.py` compare donc le PR-AUC du candidat (le run que `ml/train.py` vient de produire, identifié par le fichier marqueur `ml/artifacts/latest_training_run.json`) à celui du modèle actuellement en Production. Si le candidat est meilleur — ou si aucun modèle n'est encore en Production —, il est promu, `model.pkl` et `model_version.txt` sont commités et l'instantané `mlflow_snapshot/` servi par Komodo est republié. Sinon, il reste enregistré au stage Staging, visible et comparable dans le registre, et l'API de scoring continue de servir l'ancien modèle.

Choix assumés. Le git commit de versioning reste local au conteneur : la plateforme ne fait jamais de git push automatique, car embarquer un jeton GitHub en écriture dans un conteneur exposé sur un serveur mutualisé serait un risque disproportionné pour un projet étudiant. Les commits automatiques sont attribués à une identité dédiée (« Fraud Pipeline Bot ») plutôt qu'à un membre de l'équipe, puisque c'est la machine, et non un développeur, qui les déclenche. Enfin, si le versioning échoue, le compteur n'est pas remis à zéro : les lignes concernées sont retentées au prochain ingest au lieu de disparaître silencieusement.

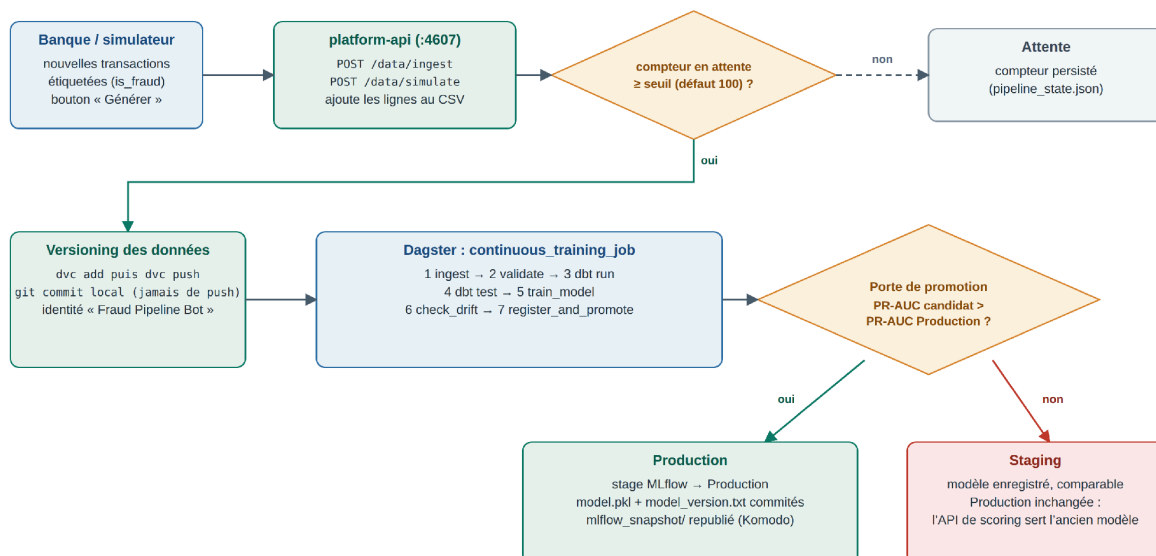


Figure 33 : Boucle d'entraînement continu — seuil de déclenchement, versioning DVC, job Dagster et porte de promotion.

5. Interface d'opérations : plateforme bancaire (React) :

La version 3 remplace le tableau de bord statique par une application monopage (dossier `platform_ui/`) écrite en React 18 et TypeScript (Vite, React Router, TanStack Query, Recharts). Elle est compilée dans la première étape de `Dockerfile.platform` (image `node:20-slim`) ; le dossier `dist` obtenu est copié dans `/opt/platform_ui_dist` et servi par FastAPI (variable `PLATFORM_UI_DIST`). Aucun service supplémentaire n'est donc nécessaire sur le serveur mutualisé : `platform-api` (port 4607) sert à la fois l'API et l'interface.

Six pages. Le menu latéral donne accès à : Transactions (table paginée, filtres, création, modification et suppression par lot), Versions du dataset (historique et rollback), Pipeline et automatisation (seuil, simulation, job suivi par étape, comparaison Production / candidat), Dérive des données (PSI par variable, historique, distributions), Historique des runs et Écosystème (santé des sept services). Une pastille rouge signale une dérive significative ou des corrections en attente.

Authentification. Les comptes sont définis par la variable d'environnement `PLATFORM_USERS` (« `utilisateur:mot_de_passe,...` ») et le jeton est un HMAC signé avec `PLATFORM_SECRET`, valable 8 heures. La vérification est faite côté API sur chaque route sauf `/health`, `/auth/login` et les fichiers statiques : masquer les boutons dans l'interface ne suffirait pas, car le service est joignable depuis Internet et expose des actions destructives (suppression de lignes, rollback, redéploiement). Le compte par défaut de démonstration (`admin:changeme`) doit être surchargé dans l'Environnement de la stack Komodo.

Intégrité des corrections. Le CSV n'a pas de clé métier : l'identifiant d'une transaction est la position de sa ligne, qui se décale après une suppression. Un verrou optimiste (`base_version` et compteur `dataset_epoch`, incrémenté à chaque correction ou rollback mais pas à un simple ajout) refuse avec un code HTTP 409 un lot préparé sur une version périmée. Les corrections sont d'abord mises en attente dans le navigateur, puis appliquées en un seul lot : réécriture atomique du CSV, `dvc add`, `dvc push` et un `commit git` attribué à l'identité « Fraud Pipeline Bot » avec l'utilisateur en mention ; si le versionnement échoue, le lot est annulé.

Rollback, dérive et promotion. Restaurer une ancienne version crée un nouveau commit : l'historique n'est jamais réécrit et rien n'est poussé vers GitHub. La dérive compare la référence (tout le dataset d'origine) aux lignes arrivées ensuite ; en dessous de 100 nouvelles lignes, l'interface retombe sur la comparaison des deux moitiés temporelles, comme `drift_report.md`. Enfin, la promotion automatique est désactivée par défaut (`auto_promote = false`) : le job enregistre le candidat en Staging et un utilisateur l'approuve ou le rejette après avoir comparé ses métriques à celles du modèle en Production.

6. Orchestration Dagster, MLflow serveur et supervision :

Orchestration Dagster. `dagster` et `platform-api` partagent le volume `dagster_home` (`DAGSTER_HOME=/dagster_home`, config dans `orchestration_dagster/dagster.yaml`) : les runs lancés

depuis le tableau de bord apparaissent dans l'UI Dagster. Un **schedule** (`nightly_pipeline_schedule`) relance `fraud_pipeline_job` chaque nuit à 02:00 (Africa/Casablanca) ; un **sensor** (`dataset_version_sensor`) relance le même job quand `data/raw/creditcard.csv.dvc` change hors plateforme (par exemple un `dvc pull` d'une autre version), sauf si cette empreinte a déjà été traitée par un run de la plateforme (`dvc_md5` dans `runs.jsonl`), pour ne pas doubler le run. Un seul run à la fois : `file_max_concurrent_runs`: 1 côté démon, plus un verrou de fichier (`data/state/pipeline.lock`) entre les étapes, car `platform-api` lance `dagster job execute` en dehors de cette file et les deux écrivent le même DuckDB.

Un serveur MLflow vivant. Avant, `mlflow` servait en lecture seule un instantané `mlflow_snapshot/mlflow.db` commité dans Git, republié à chaque ré-entraînement, avec un risque de dérive de schéma entre serveur et client (Chapitre 7, Exemple 6). Le serveur 2.22.5 (même version que `requirements.txt`) tourne désormais avec un backend PostgreSQL et des artefacts stockés dans MinIO via le proxy d'artefacts (`--serve-artifacts`). `train.py`, `register_model.py` et `platform-api` s'y connectent via `MLFLOW_TRACKING_URI=http://mlflow:5000` ; sans cette variable, le mode local SQLite est inchangé (tests, développement), et `publish_mlflow_snapshot()` est ignoré dès qu'un serveur vivant est détecté. Le registre démarre neuf — les versions `v1/v2` de l'ancien instantané n'y sont pas migrées : le premier ré-entraînement lancé depuis le tableau de bord y a enregistré une `v1` en Staging, la promotion automatique restant désactivée (`auto_promote: false`), confirmé dans l'interface MLflow (Figure 56, Chapitre 6 §9).

Supervision : dérive et alertes. `platform-api` expose `GET /metrics` (public, comme `/health`) : `fraud_drift_psi{feature}`, `fraud_drift_max_psi`, `fraud_drift_threshold`, `fraud_drift_alert`, `fraud_dataset_rows`, `fraud_pipeline_last_run_success`, entre autres. Sept règles d'alerte (`monitoring/prometheus/alerts.yml`, testées par `alerts_test.yml` en CI) couvrent trois groupes : dérive (`FraudDataDriftHigh` si le PSI max dépasse le seuil pendant 1 min, critique ; `FraudDataDriftModerate` entre 0,10 et le seuil pendant 5 min, avertissement), pipeline (`FraudPipelineLastRunFailed` si le dernier run a échoué ; `FraudPipelineStale` si aucun run depuis 3 jours) et disponibilité (`FraudApiDown`, `FraudPlatformDown`, `FraudDriftUnavailable`). Il n'y a pas d'Alertmanager : les alertes se consultent dans Prometheus (`/alerts`) et dans le panneau « Alertes en cours » du tableau de bord Grafana « Dérive & pipeline » (Figures 57 et 58, Chapitre 6 §9).

Chapitre 4 : Résultats clés

1. Indicateurs vérifiés :

Les résultats ci-dessous ont été vérifiés par exécution effective des pipelines et des tests, et non estimés. Le détail est disponible dans `docs/data/data_quality_report.md` et `docs/ml/experiments_summary.md`.

Tableau 3 : Indicateurs clés vérifiés du projet.

Indicateur	Valeur
------------	--------

Transactions ingérées (dlt → DuckDB)	284 807
Fraudes	492 (0,1727 %)
Tests qualité dbt	27/27 PASS (schéma, contenu, métier)
Modèle retenu	RandomForest (class_weight="balanced_subsample")
PR-AUC (modèle retenu / baseline)	0,8444 / 0,7200
Recall / Precision (seuil optimal)	0,8163 / 0,8421
Fraudes détectées sur le jeu de test	80 / 98
Tests automatisés (pytest)	71/71 PASS (57 précédents : 12 initiaux — API, validation, métriques ; 3 — porte de promotion ; 42 — platform_api ; 14 nouveaux : 3 — détection d'un serveur MLflow vivant ; 3 — /metrics et registre vide ; 8 — déploiement Komodo gardé par la CI et ses retries)
Endpoints API	GET /health, POST /predict, GET /metrics — testés en local et en conteneur Docker
CI/CD	Pipeline complet (lint → tests → ingestion → qualité → dbt → ML → registry → build Docker) validé dans un clone isolé
Dérive (PSI, amount / log_amount / hour_of_day)	< 0,01 — pas de dérive significative sur la période couverte (~2 jours)
Versioning du dataset (DVC)	data/raw/creditcard.csv (150 828 752 octets) suivi par DVC : pointeur .dvc versionné dans Git, contenu stocké dans le bucket MinIO dvc-store

Plateforme d'entraînement continu	Seuil configurable (100 par défaut) → dvc add/push + git commit (bot) → job Dagster en 7 étapes → porte de promotion sur le PR-AUC. Run réel sur Komodo le 19/09/2026 (3 000 lignes, dérive forte) : statut completed en ~4 min 40 s ; la porte a refusé un candidat plus faible en local, et a promu automatiquement la version 1 sur le registre vide de Komodo Interface d'opérations React authentifiée (Chapitre 3 §5, Figures 39 à 53).
Déploiement Komodo	8 services (api, mlflow, mlflow-db, grafana, prometheus, minio, platform-api, dagster) sur exp.s3.fsbm.ma, ports 4601 à 4608 ; déploiement gardé par la CI (job deploy après le job test) ; /status de la plateforme : dataset_rows = 284 807

Le suivi des expériences dans MLflow rend les chiffres du Tableau 3 directement vérifiables. La comparaison des runs (Figure 15) affiche côte à côte logreg_baseline et rf_balanced_v1, avec un PR-AUC de 0,7200 pour la baseline contre 0,8444 pour le modèle retenu. Le détail du run rf_balanced_v1 (Figure 16) expose les paramètres, métriques et artefacts (courbe PR, matrice de confusion) associés à ce résultat. Enfin, le Model Registry (Figure 17) montre le modèle fraud-detection-classifier promu au stage Production, preuve que le cycle de vie du modèle est réellement géré et non simplement documenté.

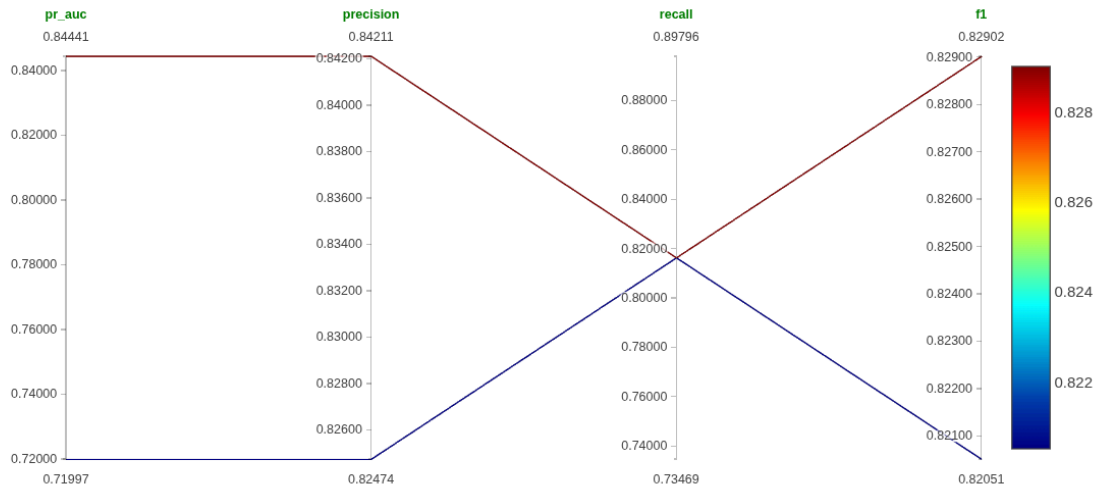


Figure 15 : Comparaison des runs MLflow — PR-AUC 0,7200 (baseline) vs 0,8444 (retenu).

Details

Created at	2026-07-10 13:33:28
Created by	kappa
Experiment ID	1 
Status	 Finished
Run ID	137b1b77a19c469888c230ca70f2b4de 
Duration	31.5s
Datasets used	—
Tags	data_version: 284807_rows code_version: aeb4122 trained_by: Aymane El Badri (ML Engineer) dbt_test_status: 23/23 PASS 
Source	 train.py  aeb4122
Logged models	 sklearn
Registered models	—

Figure 16 : Détail du run rf_balanced_v1 (paramètres, métriques, artefacts).

```
(python 3.11.13) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$
python ml/register_model.py
Meilleur run : 137b1b77a19c469888c230ca70f2b4de (pr_auc=0.8444)
Registered model 'fraud-detection-classifier' already exists. Creating a new version of this model...
Created version '2' of model 'fraud-detection-classifier'.
Modèle enregistré : fraud-detection-classifier v2
/home/kappa/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops/ml/register_model.py:56: FutureWarning: `mlflow.tracking.client.MlflowClient.transition_model_version_stage` is deprecated since 2.9.0. Model registry stages will be removed in a future major release. To learn more about the deprecation of model registry stages, see our migration guide here: https://mlflow.org/docs/latest/model-registry.html#migrating-from-stages
  client.transition_model_version_stage(
fraud-detection-classifier v2 -> Staging
/home/kappa/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops/ml/register_model.py:61: FutureWarning: `mlflow.tracking.client.MlflowClient.transition_model_version_stage` is deprecated since 2.9.0. Model registry stages will be removed in a future major release. To learn more about the deprecation of model registry stages, see our migration guide here: https://mlflow.org/docs/latest/model-registry.html#migrating-from-stages
  client.transition_model_version_stage(
fraud-detection-classifier v2 -> Production
Downloading artifacts: 100% | 7/7 [00:00<00:00, 5915.80it/s]
Modèle exporté : /home/kappa/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops/ml/artifacts/model.pkl
```

Figure 17 : Model Registry MLflow — fraud-detection-classifier au stage Production.

2. Interprétation :

Le modèle retenu améliore le PR-AUC de 17 % relatif par rapport à la baseline (0,8444 contre 0,7200), et détecte 80 fraudes sur 98 dans le jeu de test au seuil optimal, avec un compromis rappel/précision équilibré (Figure 3). L'ensemble des tests qualité de données (27/27) et des tests automatisés (71/71, dont les 12 tests initiaux de la Figure 30) passent, et l'indice PSI ne révèle pas de dérive significative sur la période couverte par le dataset.

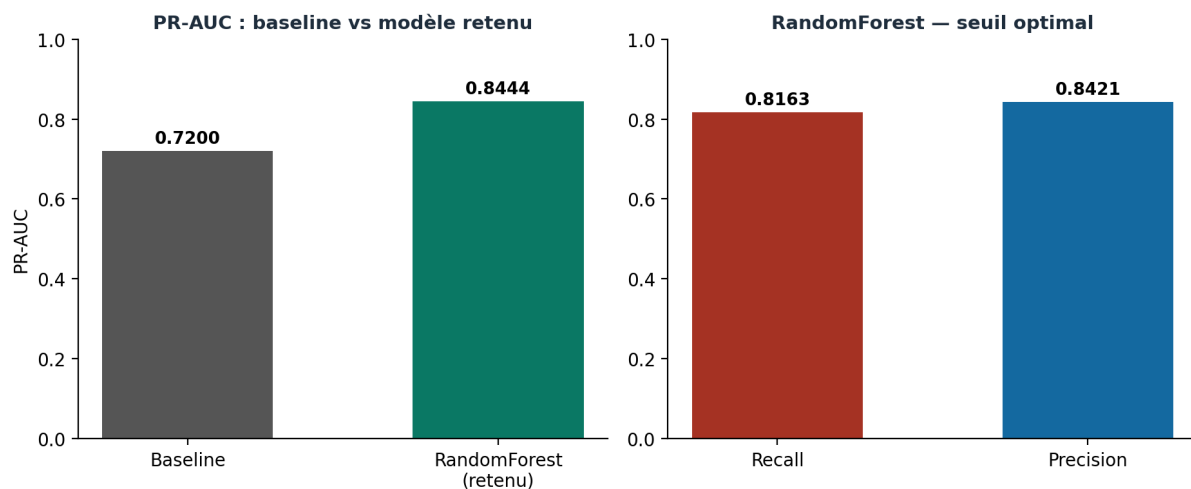


Figure 3 : PR-AUC du modèle retenu contre la baseline, et rappel/précision au seuil optimal.

Les courbes Précision-Rappel et matrices de confusion générées pour chacun des deux modèles rendent visibles les chiffres discutés ci-dessus. Pour le modèle retenu (RandomForest, class_weight="balanced_subsample"), la courbe PR (Figure 18) illustre le PR-AUC de 0,8444, et la matrice de confusion (Figure 19) le compromis rappel/précision au seuil optimal (0,8163 / 0,8421, 80 fraudes détectées sur 98). Pour comparaison, la courbe PR (Figure 20) et la matrice de confusion (Figure 21) de la baseline logistique confirment son PR-AUC nettement inférieur de 0,7200, justifiant le choix du modèle retenu.

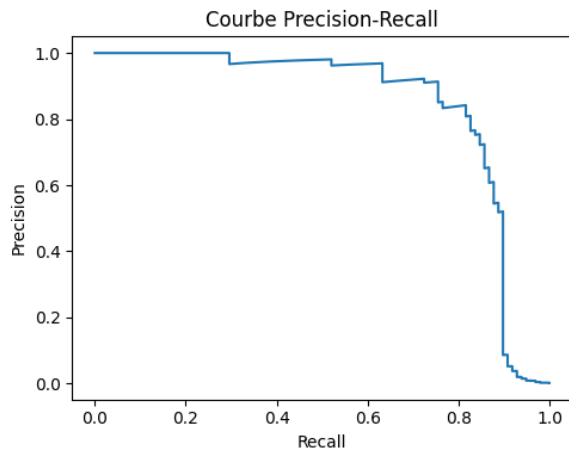


Figure 18 : Courbe Précision-Rappel — modèle retenu (rf_balanced_v1), PR-AUC = 0,8444.

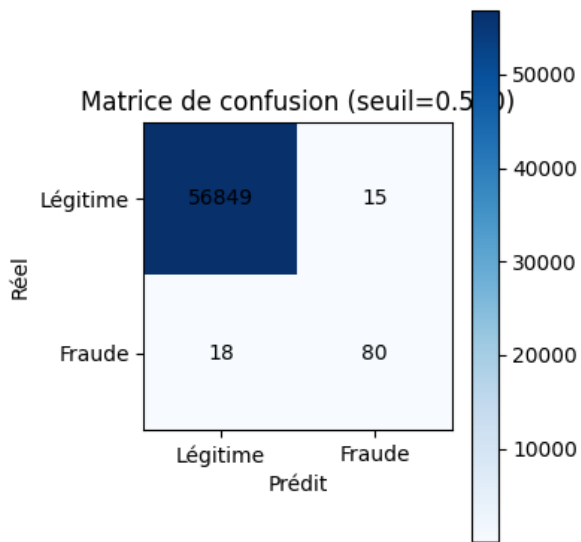


Figure 19 : Matrice de confusion — modèle retenu (recall 0,8163 / precision 0,8421).

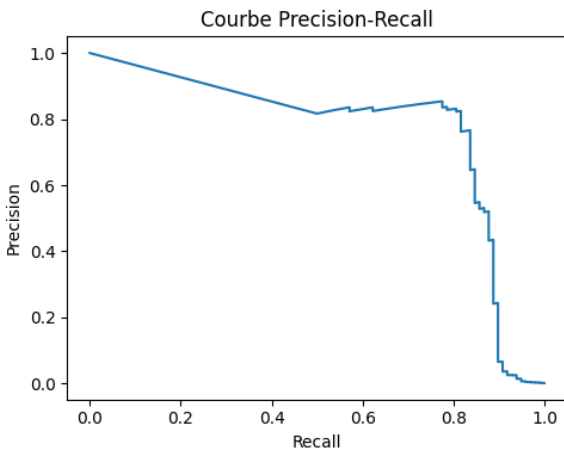


Figure 20 : Courbe Précision-Rappel — baseline (logreg_baseline), PR-AUC = 0,7200.

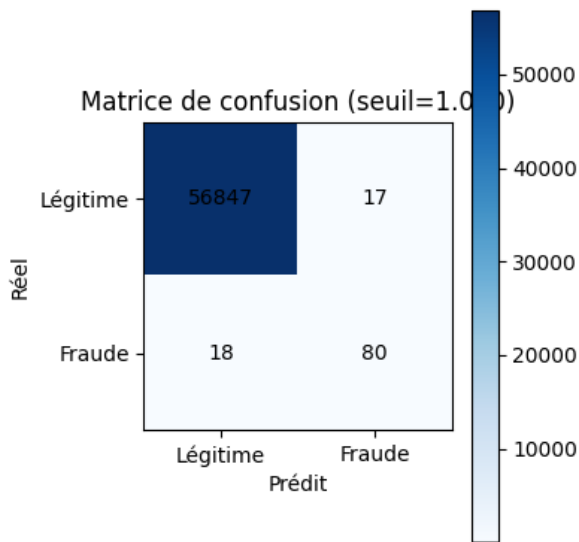


Figure 21 : Matrice de confusion — baseline (logreg_baseline).

Chapitre 5 : Mapping des 10 livrables du cahier des charges

Le Tableau 4 établit la correspondance entre chacun des 10 livrables attendus par le cahier des charges et sa réalisation effective dans le dépôt du projet.

Tableau 4 : Mapping des 10 livrables du cahier des charges.

#	Livrable	Statut	Où
1	Vision du projet	Réalisé	docs/01_vision.md

2	Gestion Agile (3 sprints)	Réalisé	docs/agile/
3	Pipeline DataOps (dlt/DuckDB/dbt/Dagster)	Réalisé	ingestion_dlt/, dbt_fraud/, orchestration_dagster/, data/raw/creditcard.csv.dvc (versioning DVC)
4	Qualité des données (Data Contract, Lineage)	Réalisé	docs/data/
5	Machine Learning	Réalisé	ml/prepare_data.py, ml/train.py, ml/evaluate.py
6	MLflow (Tracking + Registry)	Réalisé	ml/register_model.py (porte de promotion), docs/ml/experiments_summary.md
7	Déploiement (FastAPI + Docker)	Réalisé	api/, Dockerfile, docker-compose.yml (8 services), platform_api/ + platform_ui/ (interface d'opérations), déployé via Komodo (serveur vh3, exp.s3.fsbm.ma)
8	CI/CD	Réalisé	.github/workflows/ci.yml

9	Monitoring	Réalisé	monitoring/, GET /metrics, Prometheus, Grafana
10	Documentation	Réalisé	README.md, docs/02_architecture.md, docs/03_installation.md, docs/04_guide_utilisation.md

Au-delà des 10 livrables, cette version du rapport documente quatre extensions : le versioning du dataset avec DVC et MinIO (Chapitre 3, §3), la plateforme d'entraînement continu avec porte de promotion (Chapitre 3, §4), l'extension du déploiement Komodo à huit services (Chapitre 6, §6 et §7) et l'orchestration Dagster avec supervision de la dérive (Chapitre 3, §6 ; Chapitre 6, §9 et §10).

Chapitre 6 : Démonstration opérationnelle des composants livrés

Ce chapitre rassemble les preuves d'exécution réelle des composants opérationnels du projet — service de scoring, conteneurisation, intégration continue et supervision — au-delà de leur simple description dans les Tableaux 2 et 4. Chaque capture d'écran correspond à une commande effectivement exécutée dans l'environnement du projet.

1. Service de scoring (FastAPI) :

Le modèle retenu et enregistré dans MLflow (Chapitre 4) est exposé par un service FastAPI conforme aux endpoints minimaux exigés par le cahier des charges : GET /health et POST /predict, complétés par GET /metrics pour la supervision (Figure 22). Un appel réel à ces endpoints confirme que le service répond correctement et que la prédiction retournée inclut la probabilité de fraude, la décision binaire et la version du modèle chargé (Figure 23) — preuve que le modèle n'est pas seulement enregistré mais réellement chargé et utilisé pour scorer une transaction.

Fraud Detection API 1.0.0 OAS 3.1

/openapi.json

Service de scoring de transactions bancaires — projet MLOps & DataOps.

default

GET	/metrics	Metrics	▼
GET	/health	Health	▼
POST	/predict	Predict	▼

Schemas

HTTPValidationError > Expand all object

PredictionOutput > Expand all object

TransactionInput > Expand all object

ValidationError > Expand all object

Figure 22 : Documentation Swagger du service de scoring — endpoints /health, /predict, /metrics.

```
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$
curl http://localhost:8000/health
curl -X POST http://localhost:8000/predict \
-H "Content-Type: application/json" \
-d '{"time": 0, "amount": 149.62, "v1": -1.36, "v2": -0.07, "v3": 2.54, "v4": 1.38, "v5": -0.34, "v6": 0.46, "v7": 0.24, "v8": 0.10, "v9": 0.36, "v10": 0.09, "v11": -0.55, "v12": -0.62, "v13": -0.99, "v14": -0.31, "v15": 1.47, "v16": -0.47, "v17": 0.21, "v18": 0.03, "v19": 0.40, "v20": 0.25, "v21": -0.02, "v22": 0.28, "v23": -0.11, "v24": 0.07, "v25": 0.13, "v26": -0.19, "v27": 0.13, "v28": -0.02}'
{"status": "ok", "model_loaded": true, "model_version": "fraud-detection-classifier v2 (run_id=137b1b77a19c469888c230ca70f2b4de, pr_auc=0.8444, stage=Production)"}{"fraud_probability": 0.006241, "is_fraud": false, "threshold": 0.4998, "model_version": "fraud-detection-classifier v2 (run_id=137b1b77a19c469888c230ca70f2b4de, pr_auc=0.8444, stage=Production)"}(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$
```

Figure 23 : Appels curl réels sur /health et /predict, avec la réponse JSON (fraud_probability, is_fraud, model_version).

2. Conteneurisation (Docker) :

Le service est packagé dans une image Docker afin de garantir sa reproductibilité au-delà du poste de développement (Figure 24). Le même test de santé, exécuté cette fois contre le conteneur et non plus l'environnement local, démontre que le service reste fonctionnel une fois isolé (Figure 25) — une exigence de portabilité illustrée plus loin par l'incident d'ingénierie décrit au Chapitre 7.

```

(python_3.11.13) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$
docker build -t fraud-api .
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.0s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 236B 0.0s
=> [internal] load build context 0.1s
=> => transferring context: 4.51MB 0.1s
=> [1/7] FROM docker.io/library/python:3.11-slim@sha256:e031123e3d85762b141ad1cb56452ba69c6e722ebf2f042cc0dc86c47c0d8b3 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:e031123e3d85762b141ad1cb56452ba69c6e722ebf2f042cc0dc86c47c0d8b3 0.0s
=> CACHED [2/7] WORKDIR /app 0.0s
=> CACHED [3/7] COPY requirements-api.txt . 0.0s
=> CACHED [4/7] RUN pip install --no-cache-dir -r requirements-api.txt 0.0s
=> [5/7] COPY api/ api/ 0.0s
=> [6/7] COPY ml/artifacts/model.pkl ml/artifacts/model.pkl 0.0s
=> [7/7] COPY ml/artifacts/model_version.txt ml/artifacts/model_version.txt 0.0s
=> exporting to image 4.8s
=> => exporting layers 0.3s
=> => exporting manifest sha256:49c63ac075ed7831c130fcbdb083fed02c988ce39a35b509c852af8e926a34 0.0s
=> => exporting config sha256:a8fc58f072ecb85c5b42fc0265e3c5b63a378alea82b8eae193a9dfaf20d6a 0.0s
=> => exporting attestation manifest sha256:563eba233271aa91ecb51ba2ab375481c50813b0287d50d8331becc51665fdb 0.0s
=> => exporting manifest list sha256:8393a8d67d65bd86ff4ebdb9722a82154dce23406a46dc8d4f53ed272838be2a 0.0s
=> => naming to docker.io/library/fraud-api:latest 0.0s
=> => unpacking to docker.io/library/fraud-api:latest 4.3s
(python_3.11.13) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$
  
```

Figure 24 : Fin du build Docker (image fraud-api taguée avec succès).

```

4.3s
(python_3.11.13) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$ docker run -p 8090:8000 fraud-api
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 172.17.0.1:33844 - "GET /health HTTP/1.1" 200 OK
INFO: 172.17.0.1:33844 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:38210 - "GET /health HTTP/1.1" 200 OK
INFO: 172.17.0.1:33828 - "GET / HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:45314 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:55988 - "GET /health HTTP/1.1" 200 OK
INFO: 172.17.0.1:38574 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:53586 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:39276 - "GET /health HTTP/1.1" 200 OK
  
```

```

{
  "status": "ok",
  "model_loaded": true,
  "model_version": "fraud-detection-classifier v2
stage=Production)"
}
  
```

Figure 25 : Appel curl /health exécuté contre le conteneur Docker, confirmant la portabilité du service.

3. Intégration et déploiement continu (CI/CD) :

L'ensemble du pipeline — lint, tests, ingestion, qualité, dbt, ML, registry, build Docker — est rejoué automatiquement à chaque push via GitHub Actions (Tableau 3), et non simplement décrit : la Figure 26 montre un run récent entièrement vert, preuve indépendante de l'environnement local que le Livrable 8 (CI/CD) est opérationnel.

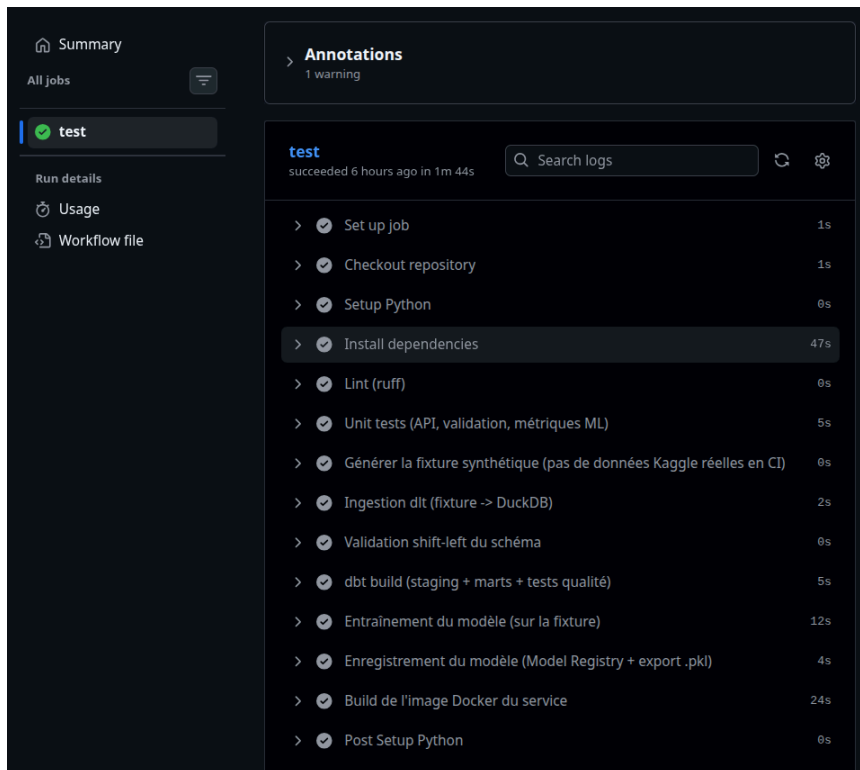


Figure 26 : Run GitHub Actions complet, toutes les étapes passées avec succès.

4. Supervision et détection de dérive :

Le service expose des métriques au format Prometheus (compteurs de requêtes, latence), consultables via GET /metrics (Figure 27). La détection de dérive, basée sur l'indice PSI calculé sur les variables amount, log_amount et hour_of_day entre les deux moitiés temporelles du jeu de référence (142 403 vs 142 404 transactions), confirme l'absence de dérive significative ($PSI < 0,01$, Tableau 3), comme le montre l'exécution du script de contrôle (Figure 28) et le rapport de dérive généré (Figure 29). Ce même rapport compare également la distribution de référence aux prédictions réelles journalisées par l'API : sur les 2 seules requêtes enregistrées à ce stade, le PSI mesuré (12,45) est signalé par l'outil comme une dérive potentielle, mais un échantillon de cette taille ne permet aucune conclusion fiable — cet indicateur devra être réévalué une fois un volume de trafic réel suffisant accumulé. Ces résultats doivent par ailleurs être relativisés : comme discuté au Chapitre 8, le jeu de données historique ne couvre qu'environ deux jours, ce qui limite la portée de toute conclusion de dérive en conditions réelles de production.

```

localhost:8090/metrics

# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 519.0
python_gc_objects_collected_total{generation="1"} 398.0
python_gc_objects_collected_total{generation="2"} 0.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 365.0
python_gc_collections_total{generation="1"} 33.0
python_gc_collections_total{generation="2"} 3.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="11",patchlevel="15",version="3.11.15"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 2.021216256e+09
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 1.84946688e+08
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.7836985798e+09
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 7.13
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 10.0
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1024.0
# HELP http_requests_total Total number of requests by method, status and handler.
# TYPE http_requests_total counter
http_requests_total{handler="/health",method="GET",status="2xx"} 18.0
http_requests_total{handler="none",method="GET",status="4xx"} 2.0
# HELP http_requests_created Total number of requests by method, status and handler.
# TYPE http_requests_created gauge

```

Figure 27 : Sortie Prometheus de l'endpoint /metrics du service en fonctionnement.

```

(python 3.11.13) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$
curl http://localhost:8000/metrics
python monitoring/drift_check.py
http_request_duration_seconds_bucket{handler="/health",le="+Inf",method="GET"} 2.0
http_request_duration_seconds_count{handler="/health",method="GET"} 2.0
http_request_duration_seconds_sum{handler="/health",method="GET"} 0.017854427998827305
http_request_duration_seconds_bucket{handler="/predict",le="0.1",method="POST"} 1.0
http_request_duration_seconds_bucket{handler="/predict",le="0.5",method="POST"} 2.0
http_request_duration_seconds_bucket{handler="/predict",le="1.0",method="POST"} 2.0
http_request_duration_seconds_bucket{handler="/predict",le="+Inf",method="POST"} 2.0
http_request_duration_seconds_count{handler="/predict",method="POST"} 2.0
http_request_duration_seconds_sum{handler="/predict",method="POST"} 0.16438842800198472
# HELP http_request_duration_seconds_created Latency with only few buckets by handler. Made to be only used if aggregation by handler is important.
# TYPE http_request_duration_seconds_created gauge
http_request_duration_seconds_created{handler="none",method="GET"} 1.7836982274915743e+09
http_request_duration_seconds_created{handler="/docs",method="GET"} 1.7836982365072072e+09
http_request_duration_seconds_created{handler="/openapi.json",method="GET"} 1.7836982370243855e+09
http_request_duration_seconds_created{handler="/health",method="GET"} 1.7836983069728699e+09
http_request_duration_seconds_created{handler="/predict",method="POST"} 1.783698343352965e+09
Colonne      PSI      Interprétation
amount       0.0027   pas de dérive significative
log_amount   0.0027   pas de dérive significative
hour_of_day  0.0034   pas de dérive significative

amount (API live) 12.4461 dérive significative - ré-entraînement à envisager (n=2)

Rapport écrit : /home/kappa/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops/monitoring/drift_report.md
(python 3.11.13) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$

```

Figure 28 : Sortie terminal du contrôle de dérive (PSI par variable).

Référence : 142403 transactions (1ère moitié temporelle) — Récent : 142404 transactions (2nde moitié temporelle).

Colonne	PSI	Interprétation
amount	0.0027	pas de dérive significative
log_amount	0.0027	pas de dérive significative
hour_of_day	0.0034	pas de dérive significative

Comparaison avec 2 prédictions réelles journalisées par l'API (`monitoring/predictions_log.jsonl`) :

Colonne	PSI	Interprétation
amount (trafic API)	12.4461	dérive significative — ré-entraînement à envisager

Figure 29 : Extrait du rapport de dérive généré (monitoring/drift_report.md).

5. Tests automatisés :

La couverture de tests unitaires citée au Tableau 3 (12/12 à l'origine) est vérifiée par exécution effective de la suite pytest, couvrant l'API, la validation des données et les métriques ML (Figure 30). Cette capture correspond à la suite initiale ; la version 3 avait porté la suite à 57 tests (3 pour la porte de promotion, 42 pour platform_api) ; la version 4 y ajoute 14 tests : 3 pour la détection d'un serveur MLflow vivant depuis le schéma de l'URI (ml/register_model.py, Chapitre 7 Exemple 7), 3 pour /metrics et la comparaison sur un registre vide (platform_api), et 8 pour le déploiement Komodo gardé par la CI et ses retries (scripts/komodo_deploy.py, Chapitre 7 Exemple 8) — 71 tests au total, tous passants (71 passed en 15,8 s lors de la dernière exécution, le 22 septembre 2026).

```

(python 3_11_13) kappa@kappa-vivo:~/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops$
• pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.11.13, pytest-9.0.2, pluggy-1.6.0 -- /home/kappa/miniconda3/envs/python_3_11_13/bin/python
cachedir: .pytest_cache
rootdir: /home/kappa/Documents/etude/master/S2/mlops_dataops/fraud-detection-mlops
configfile: pytest.ini
plugins: dash-4.1.0, anyio-4.12.1
collected 12 items

tests/test_api.py::test_health_and_predict_happy_path PASSED [ 8%]
tests/test_api.py::test_predict_rejects_incomplete_payload PASSED [ 16%]
tests/test_api.py::test_predict_rejects_negative_amount PASSED [ 25%]
tests/test_api.py::test_metrics_endpoint_exposed PASSED [ 33%]
tests/test_ml_pipeline.py::test_compute_metrics_perfect_separation PASSED [ 41%]
tests/test_ml_pipeline.py::test_compute_metrics_confusion_counts PASSED [ 50%]
tests/test_ml_pipeline.py::test_best_threshold_by_f1_returns_valid_probability PASSED [ 58%]
tests/test_validate_schema.py::test_validate_passes_on_clean_data PASSED [ 66%]
tests/test_validate_schema.py::test_validate_fails_on_missing_column PASSED [ 75%]
tests/test_validate_schema.py::test_validate_fails_on_null_amount PASSED [ 83%]
tests/test_validate_schema.py::test_validate_fails_on_invalid_class_domain PASSED [ 91%]
tests/test_validate_schema.py::test_validate_fails_on_negative_amount PASSED [100%]

```

Figure 30 : Résultat final de la suite de tests pytest — 12 passed (suite initiale).

6. Déploiement continu sur Komodo :

Au-delà de l'exécution locale documentée aux sections précédentes, le projet est déployé sur un serveur dédié via **Komodo** (<https://komodo.s3.fsbm.ma>), une plateforme d'orchestration Docker fournie dans le cadre du module. Le dépôt Git (kappa20/fraud-detection-mlops, branche main) est cloné et reconstruit par Komodo sur le serveur vh3 : chaque redéploiement exécute git pull, docker compose build puis docker compose up. L'option « Pre Build Images » est activée pour que l'image soit reconstruite à chaque déploiement, et un webhook GitHub relance le déploiement à chaque push sur main. Les services applicatifs sont exposés sur le domaine exp.s3.fsbm.ma.

Le stack, initialement composé de trois services (api, prometheus, grafana), en compte désormais huit (Figure 31, Tableau 6, Figures 35, 36 et 38 pour les six premiers ; Chapitre 6 §9 pour Dagster et le serveur MLflow), tous rattachés au réseau Docker fraud_net et configurés avec restart: unless-stopped. MLflow, servi jusqu'à la version 3 en lecture seule à partir d'un instantané mlflow_snapshot/ republié à chaque ré-entraînement, est désormais un serveur vivant (PostgreSQL, artefacts MinIO) partagé avec l'orchestrateur Dagster (Chapitre 3, §6).

Tableau 6 : Services du stack déployé sur Komodo.

Service	Rôle	Port hôte
api	API de scoring FastAPI (/health, /predict, /metrics, /docs)	4601
mlflow	Serveur MLflow vivant (tracking et registre), backend PostgreSQL, artefacts sur MinIO	4602
grafana	Tableaux de bord, datasource Prometheus provisionnée	4603
prometheus	Collecte périodique de /metrics de l'API	4604
minio	Stockage objet S3 : remote DVC (bucket dvc-store)	4605 (API), 4606 (console)
platform-api	Plateforme d'entraînement continu et interface d'opérations (React)	4607
dagster	UI d'orchestration : graphe des jobs, schedule nocturne (02:00), sensor sur le dataset	4608
mlflow-db	Base PostgreSQL du serveur MLflow (backend du tracking et du registre)	— (interne)

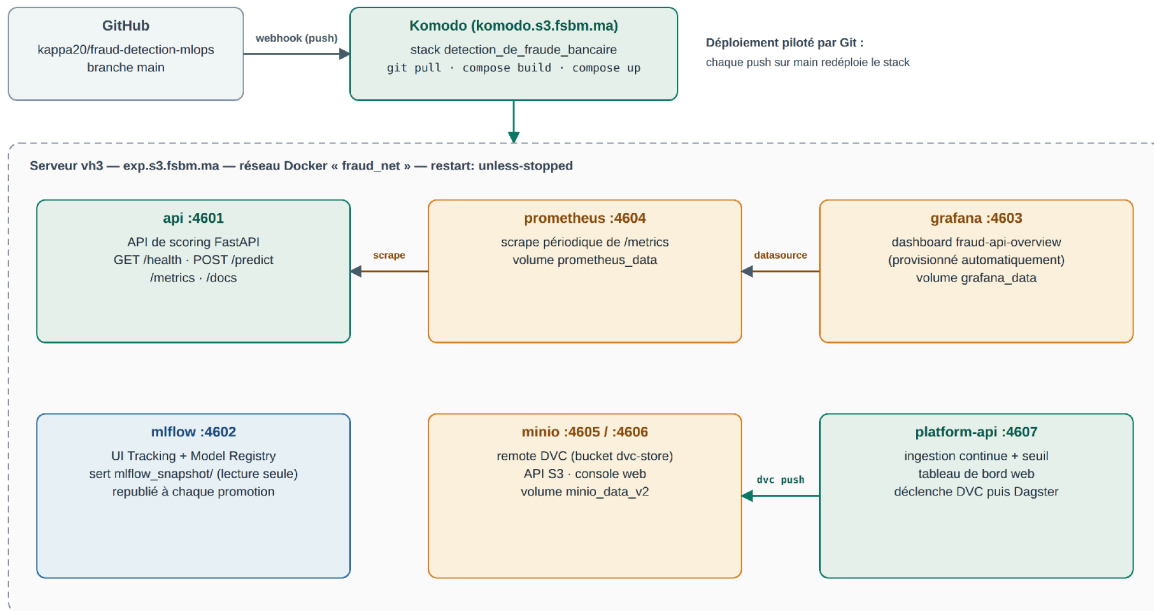


Figure 35 : Architecture de déploiement sur Komodo — six services sur le réseau fraud_net, redéployés par webhook GitHub.

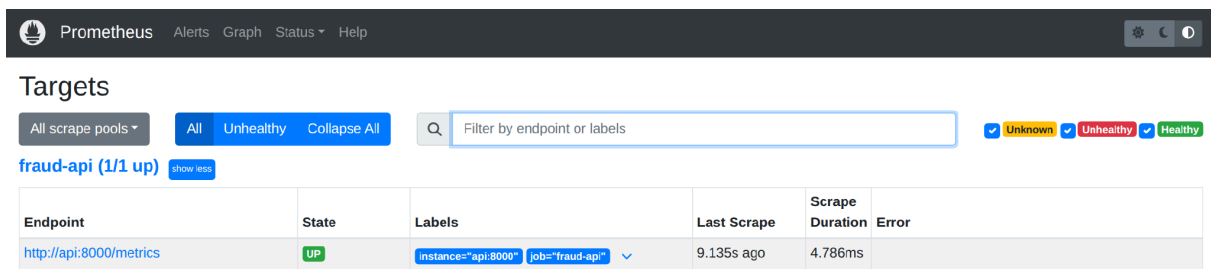


Figure 36 : Interface Prometheus déployée sur Komodo (exp.s3.fsbm.ma:4604) — cible fraud-api (api:8000/metrics) à l'état UP, 1/1 up, capture du 19 septembre 2026.

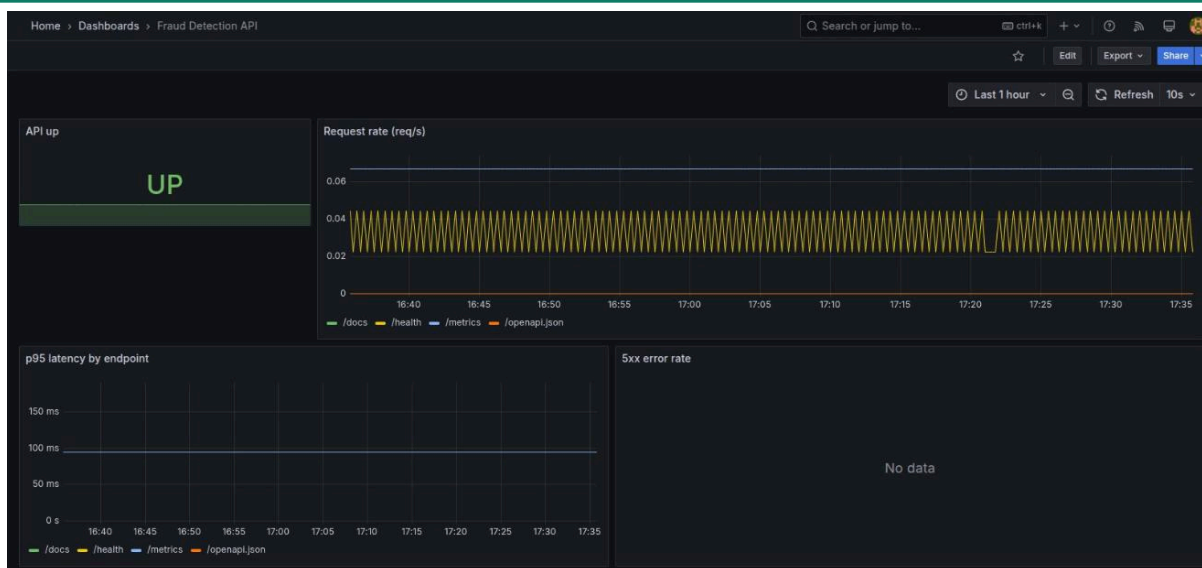


Figure 38 : Tableau de bord Grafana « Fraud Detection API » (exp.s3.fsbm.ma:4603), capture du 19 septembre 2026 : API up, taux de requêtes par endpoint (/health, /metrics, /docs, /openapi.json), latence p95 par endpoint (environ 100 ms pour /metrics) et taux d’erreurs 5xx sans donnée (aucune erreur).

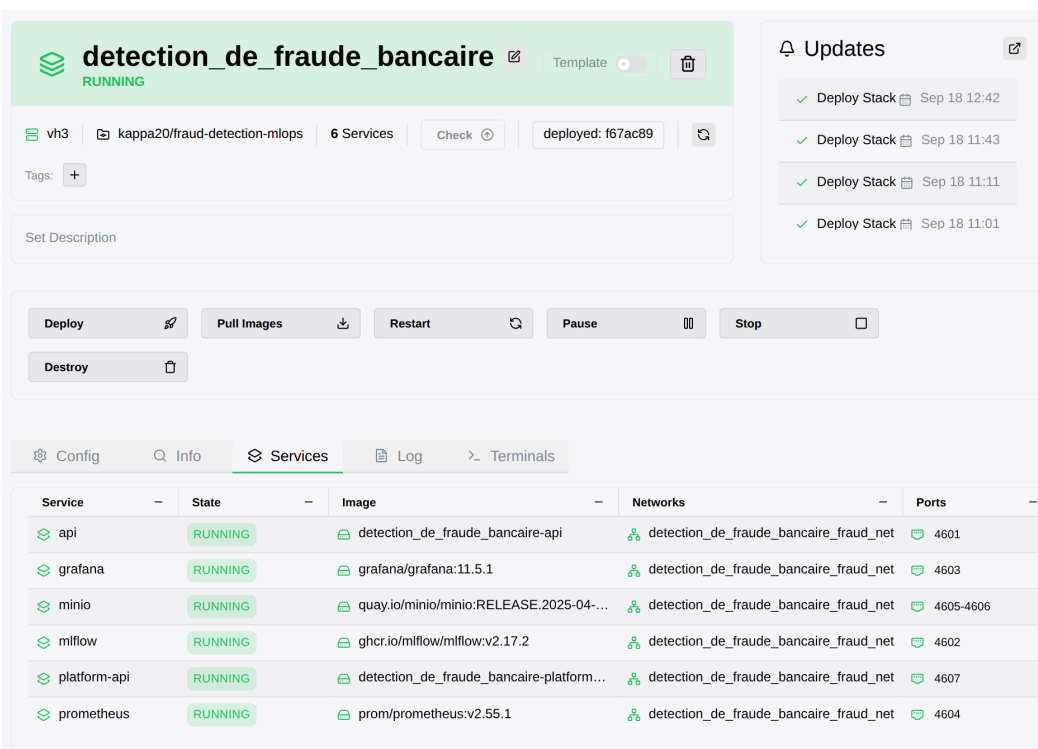


Figure 31 : Interface Komodo, stack detection_de_fraude_bancaire à l’état RUNNING sur le serveur vh3 (commit déployé f67ac89) : les six services api, grafana, minio, mlflow, platform-api et prometheus sont RUNNING, avec leurs ports 4601 à 4607. Capture du 19 septembre 2026.

Gestion des ports. Lors du premier déploiement, l'API échouait avec l'erreur port is already allocated : le port 8000 par défaut était déjà utilisé par le projet d'une autre équipe sur ce serveur mutualisé. Les ports avaient alors été rendus paramétrables par variables d'environnement (API_HOST_PORT, PROMETHEUS_HOST_PORT, GRAFANA_HOST_PORT). Un schéma de ports fixe, dans la plage 4601–4607 (Tableau 6), a ensuite remplacé ces variables dans docker-compose.yml, ce qui évite les collisions sans dépendre de la configuration saisie dans l'interface Komodo.

Limites assumées. (1) Le fichier réel du dataset n'étant pas dans le dépôt Git, il doit être récupéré une fois par nouveau clone sur Komodo (dvc pull, lancé depuis le terminal du conteneur platform-api) : le webhook ne le fait pas. (2) Les identifiants par défaut de MinIO (admin / admin12345) n'ont pas été renouvelés : la console MinIO les accepte encore (vérifié le 19 septembre 2026), alors que ceux de Grafana ont été modifiés (les identifiants par défaut sont refusés). Sur un serveur mutualisé accessible depuis Internet, la rotation des identifiants MinIO via les variables d'environnement du stack est à réaliser avant tout usage au-delà de la démonstration. (3) La limite signalée dans la première version de ce rapport (délais de connexion sur les ports 8001, 9096 et 3009) ne s'est plus reproduite avec le schéma actuel : un des tests avait notamment utilisé une adresse IP erronée pour le serveur, et l'accès externe a été vérifié le 19 septembre 2026 : les sept points d'accès (ports 4601 à 4607 de exp.s3.fsbm.ma) répondent HTTP 200, et GET /status de la plateforme retourne dataset_rows égal à 284 807.

7. Plateforme d'entraînement continu en fonctionnement :

La plateforme a été exécutée de bout en bout sur le serveur Komodo le 19 septembre 2026, avec l'accord de l'équipe, par un appel POST /data/simulate (3 000 transactions, dérive strong). Le dataset est passé de 284 807 à 287 807 lignes, a reçu une nouvelle empreinte DVC (9199c2c9..., poussée dans MinIO : Figure 37) et un commit local de l'identité bot (fbd34a96) ; le job Dagster complet (ingestion, dbt, entraînement, dérive, enregistrement) s'est ensuite terminé avec le statut completed en environ 4 min 40 s (Figure 34). Les entrées de ce run sont visibles dans la page « Historique des runs » de l'interface d'opérations de l'instance Komodo (Figure 34) ; cette interface est présentée en détail au paragraphe suivant.

Résultat de la porte de promotion. Le registre MLflow du conteneur platform-api était vide : mlflow.db n'est pas versionné dans Git, donc le dépôt cloné par Komodo n'en contient aucun. La porte n'avait aucun modèle en Production à battre et a promu automatiquement le candidat comme version 1 (PR-AUC 0,7592, entraîné sur le dataset augmenté de lignes à dérive forte, contre 0,8444 pour le modèle historique). Le service de scoring, qui charge son modèle au démarrage, n'était pas affecté à la date de la capture (/health : version 2, PR-AUC 0,8444). Ce comportement est conforme à la règle implémentée, mais il en révèle la limite : la porte ne protège que si le registre contient déjà un modèle de référence (Chapitres 7 et 8). Le refus d'un candidat plus faible a, lui, été observé lors des essais en environnement local.

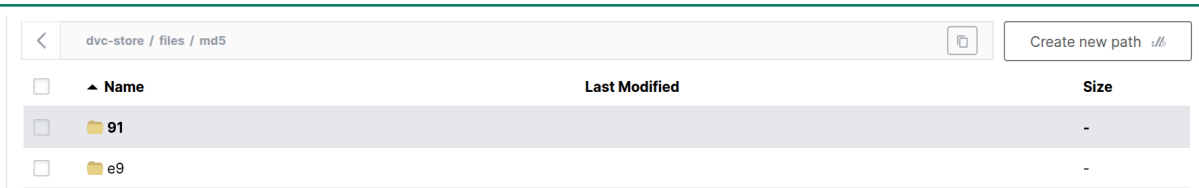


Figure 37 : Console MinIO (exp.s3.fsbm.ma:4606), bucket dvc-store, dossier files/md5/ : deux préfixes, e9... (dataset d'origine, md5 e90efcb8...) et 91... (version produite par le run de démonstration, md5 9199c2c9...).

HISTORIQUE DES RUNS							Statut	Tous
HORODATAGE	DÉCLENCHEUR	LIGNES	STATUT	DATASET (DVC)	COMMIT	NOTE		
19/09/2026 15:55:38	Simulation	3 000	Terminé	9199c2c9	fbd34a96	Pipeline complet exécuté (ingestion, transformation, tests, entraînement, dérive, regist...		Relancer
19/09/2026 15:50:58	Simulation	3 000	En cours	9199c2c9	fbd34a96	Dataset versionné (DVC/git) : pipeline complet (entraînement, dérive, registry) en cours.		Relancer

Figure 34 : Page « Historique des runs » de la plateforme d'opérations, instance déployée sur Komodo (exp.s3.fsbm.ma:4607) : les deux entrées du run de démonstration du 19 septembre 2026 (3 000 lignes simulées, empreinte DVC 9199c2c9, commit fbd34a96). Ces entrées ont été écrites par la version précédente de la plateforme ; la première, enregistrée au moment du versionnement, conserve son statut d'origine « En cours ».

Pour une démonstration, il faut générer plusieurs milliers de lignes avec une intensité strong : quelques lignes seulement ne déplacent pas le PSI agrégé face aux 284 807 lignes de référence. Cette exécution réelle a par ailleurs révélé trois défauts que les tests unitaires (avec mocks) n'avaient pas détectés ; ils sont détaillés au Chapitre 7.

8. Interface d'opérations en fonctionnement :

Les figures de ce paragraphe proviennent de deux sources, indiquées dans chaque légende. L'**instance Komodo** (exp.s3.fsbm.ma:4607) n'a été consultée qu'en lecture seule : aucune action d'écriture n'y a été déclenchée, conformément à la règle fixée après le run du 19 septembre. Les flux qui écrivent dans Git, DVC et MLflow ont été exécutés sur une **instance locale de démonstration** : clone jetable du dépôt, jeu de données complet de 284 807 lignes, remote DVC dans un dossier local (rien n'est poussé vers MinIO) et registre MLflow restauré depuis la sauvegarde antérieure à la migration de schéma (Chapitre 7, Exemple 6).

Connexion (Figures 39 et 40). La page de connexion de l'instance Komodo (Figure 39) est la seule page servie sans jeton. Un mot de passe erroné est refusé par l'API elle-même (Figure 40), et non par un simple contrôle dans le navigateur.

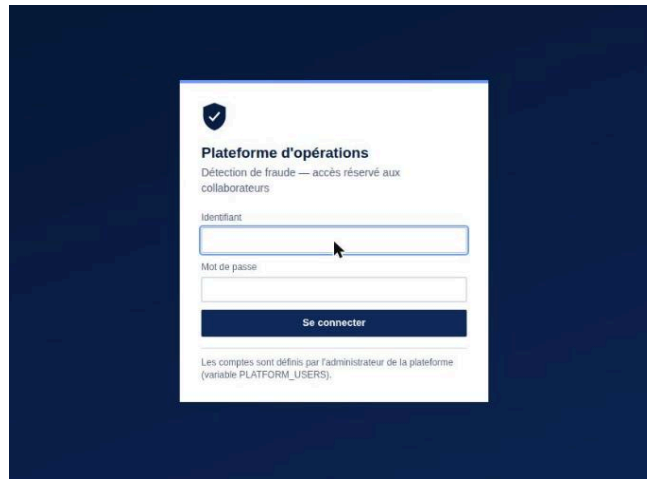


Figure 39 : Page de connexion de la plateforme d'opérations, instance déployée sur Komodo (exp.s3.fsbm.ma:4607).

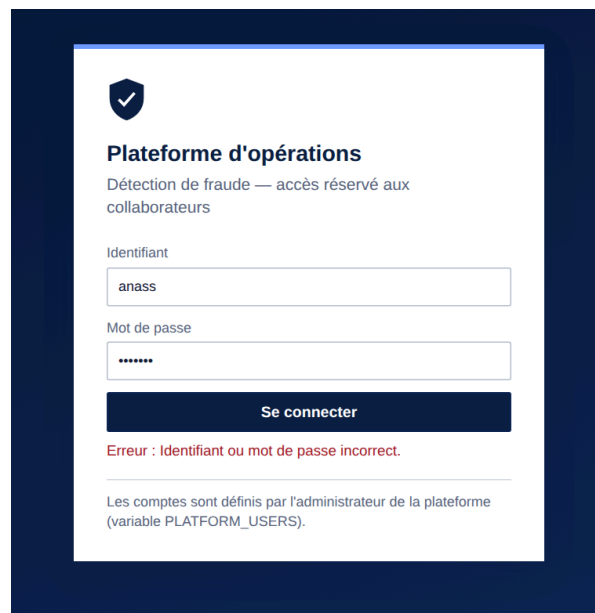


Figure 40 : Identifiants erronés : l'API refuse la connexion et l'interface affiche « Identifiant ou mot de passe incorrect » (instance locale de démonstration).

Transactions et corrections versionnées (Figures 41 à 44). La table (Figure 41) est paginée et filtrée côté serveur : le filtre « Fraude » retourne les 492 fraudes du jeu de données, et une ligne dépliée affiche les composantes anonymisées V1 à V28. « Modifier » ouvre un tiroir d'édition (Figure 42). La correction n'est pas envoyée immédiatement : elle est mise en attente (Figure 43 — ligne surlignée, pastille dans le menu, barre « 1 modification(s) en attente »). « Enregistrer... » ouvre une confirmation qui récapitule le lot (Figure 44) ; l'enregistrement applique le lot au CSV, exécute dvc add et dvc push, puis crée un commit git attribué à « Fraud Pipeline Bot » avec l'utilisateur en mention.

Détection de fraude
Plateforme d'opérations

- Transactions
- Versions du dataset
- Pipeline & automatisation
- Dérive des données
- Historique des runs
- Ecosystème

Connecté : anass Déconnexion

TRANSACTIONS
284 807
dans le dataset versionné

EN ATTENTE DU PIPELINE
0 / 100
lignes ajoutées depuis le dernier run

FILTREES
492
correspondent aux critères

CORRECTIONS EN ATTENTE
0
non enregistrées

FILTRES

Montant min Montant max Label Du jour n° (J1 = début du jeu) Au jour n°

Réinitialiser Appliquer

TRANSACTIONS + Nouvelle transaction

TRANSACTION	TEMPS	MONTANT	LABEL	CHARGEMENT	ACTIONS	
☐ TX-000541	J1 00:06:46	0,00	Fraude	-	Voir Modifier Supprimer	
V1 -2.31223 V8 1.39166 V15 0.38972 V22 -0.83505	V2 1.95199 V9 -2.77089 V16 -1.14075 V23 -0.46521	V3 -1.60985 V10 -2.77227 V17 -2.83086 V24 0.32829	V4 3.99791 V11 3.28263 V18 -0.01662 V25 0.84452	V5 -0.52219 V12 -2.89991 V19 0.41696 V26 0.17784	V6 -1.42655 V13 -0.59522 V20 0.12691 V27 0.26115	V7 -2.53739 V14 -4.28925 V21 0.51723 V28 -0.14328
☐ TX-000623	J1 00:07:52	529,00	Fraude	-	Voir Modifier Supprimer	
☐ TX-004920	J1 01:14:22	239,93	Fraude	-	Voir Modifier Supprimer	
☐ TX-006108	J1 01:56:26	59,00	Fraude	-	Voir Modifier Supprimer	
☐ TX-006329	J1 02:05:19	1,00	Fraude	-	Voir Modifier Supprimer	
☐ TX-006331	J1 02:05:26	1,00	Fraude	-	Voir Modifier Supprimer	
☐ TX-006334	J1 02:05:35	1,00	Fraude	-	Voir Modifier Supprimer	
☐ TX-006336	J1 02:05:43	1,00	Fraude	-	Voir Modifier Supprimer	
☐ TX-006338	J1 02:05:51	1,00	Fraude	-	Voir Modifier Supprimer	
☐ TX-006427	J1 02:06:50	1,00	Fraude	-	Voir Modifier Supprimer	

Figure 41 : Transactions — filtre « Fraude » (492 lignes sur 284 807) et composantes V1–V28 de la première ligne dépliée (instance locale de démonstration).

Détection de fraude
Plateforme d'opérations

- Transactions
- Versions du dataset
- Pipeline & automatisation
- Dérive des données
- Historique des runs
- Ecosystème

Connecté : anass Déconnexion

TRANSACTIONS
284 807
dans le dataset versionné

EN ATTENTE DU PIPELINE
0 / 100
lignes ajoutées depuis le dernier run

FILTREES
492
correspondent aux critères

CORRECTIONS EN ATTENTE
0
non enregistrées

FILTRES

Montant min Montant max Label Du jour n° (J1 = début du jeu) Au jour n°

Réinitialiser Appliquer

TRANSACTIONS + Nouvelle transaction

TRANSACTION	TEMPS	MONTANT	LABEL	CHARGEMENT	ACTIONS
☐ TX-000541	J1 00:06:46	0,00	Fraude	-	Voir Modifier Supprimer
☐ TX-000623	J1 00:07:52	529,00	Fraude	-	Voir Modifier Supprimer
☐ TX-004920	J1 01:14:22	239,93	Fraude	-	Voir Modifier Supprimer
☐ TX-006108	J1 01:56:26	59,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006329	J1 02:05:19	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006331	J1 02:05:26	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006334	J1 02:05:35	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006336	J1 02:05:43	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006338	J1 02:05:51	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006427	J1 02:06:50	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006446	J1 02:07:52	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006472	J1 02:09:00	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006529	J1 02:11:31	1,00	Fraude	-	Voir Modifier Supprimer

TX-000623 — Modifier

Temps (secondes depuis le début du jeu) Montant

Label

Composantes PCA anonymisées (V1–V28)

La modification n'est envoyée qu'à « Enregistrer » (un commit DVC pour tout le lot).

Annuler Mettre en attente

Figure 42 : Tiroir d'édition de la transaction TX-000623 : le montant passe de 529,00 à 545,00 ; rien n'est encore envoyé (instance locale de démonstration).

Détection de fraude
Plateforme d'opérations

Connecté : anass | Déconnexion

Transactions 1

- Versions du dataset
- Pipeline & automatisation
- Dérive des données
- Historique des runs
- Ecosystème

TRANSACTIONS
284 807
dans le dataset versionné

EN ATTENTE DU PIPELINE
0 / 100
lignes ajoutées depuis le dernier run

FILTREES
492
correspondent aux critères

CORRECTIONS EN ATTENTE
1
non enregistrées

FILTRES Réinitialiser Appliquer

Montant min: Montant max: Label: Du jour n° (J1 = début du jeu): Au jour n°:

TRANSACTIONS + Nouvelle transaction

TRANSACTION	TEMPS	MONTANT	LABEL	CHARGEMENT	ACTIONS
☐ TX-000541	J1 00:06:46	0,00	Fraude	-	Voir Modifier Supprimer
☐ TX-000623	J1 00:07:52	\$45,00	Fraude	-	Voir Modifier Supprimer
☐ TX-004920	J1 01:14:22	239,93	Fraude	-	Voir Modifier Supprimer
☐ TX-006108	J1 01:56:26	59,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006329	J1 02:05:19	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006331	J1 02:05:26	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006334	J1 02:05:35	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006336	J1 02:05:43	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006338	J1 02:05:51	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006427	J1 02:06:50	1,00	Fraude	-	Voir Modifier Supprimer
☐ TX-006446	J1 02:07:52	1,00	Fraude	-	Voir Modifier Supprimer

1 modification(s) en attente — 0 création(s), 1 mise(s) à jour, 0 suppression(s)

Tout annuler Enregistrer...

Figure 43 : La correction est en attente : ligne surlignée, pastille « 1 » sur l'entrée Transactions du menu, barre « Tout annuler / Enregistrer... ».

Détection de fraude
Plateforme d'opérations

Connecté : anass | Déconnexion

Transactions 1

- Versions du dataset
- Pipeline & automatisation
- Dérive des données
- Historique des runs
- Ecosystème

TRANSACTIONS
284 807
dans le dataset versionné

EN ATTENTE DU PIPELINE
0 / 100
lignes ajoutées depuis le dernier run

FILTREES
492
correspondent aux critères

CORRECTIONS EN ATTENTE
1
non enregistrées

FILTRES Réinitialiser Appliquer

Montant min: Montant max: Label: Du jour n° (J1 = début du jeu): Au jour n°:

TRANSACTIONS + Nouvelle transaction

Enregistrer les modifications

Ce lot sera appliqué au dataset, puis versionné avec dvc add et un commit git. Le hash du commit sera affiché et journalisé.

Créations	0
Modifications	1
Suppressions	0

Retour Enregistrer et versionner

1 modification(s) en attente — 0 création(s), 1 mise(s) à jour, 0 suppression(s)

Tout annuler Enregistrer...

Figure 44 : Confirmation « Enregistrer les modifications » : 0 création, 1 modification, 0 suppression ; le lot sera versionné avec dvc add et un commit git.

Pipeline suivi étape par étape (Figures 45 et 46). Une simulation de 3 000 transactions à dérive forte a dépassé le seuil de 100 lignes, ce qui a déclenché le versionnement puis le job Dagster continuous_training_job. La page affiche les sept étapes (ingestion, validation, transformation, tests, entraînement, contrôle de dérive, registry), alimentées par les lignes STEP_START et STEP_SUCCESS du

journal Dagster (Figure 45). Le job s'est terminé en environ 3 min 30 s sur la machine de développement (Figure 46). Le candidat produit (version 3, Staging) est nettement moins bon que le modèle en Production : PR-AUC 0,6771 contre 0,8444 et rappel 0,7290 contre 0,8163. Ce résultat est attendu, puisqu'il a été entraîné sur un jeu contenant 3 000 lignes synthétiques à dérive forte ; comme la promotion automatique est désactivée, il attend une décision humaine.

DERNIER JOB

En cours

Simulation · 19/09/2026 22:04:38 · anass

Ingestion

Validation

Transformation

Tests

Entraînement

Contrôle de dérive

Registry

▼ Journal d'exécution

```

z[0m21:06:45
z[0m21:06:45 Finished running 3 table models, 1 view model in 0 hours 0 minutes and 0.89 seconds (0.89s).
z[0m21:06:46 z[32mCompleted successfullyz[0m
z[0m21:06:46 Done. PASS=4 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=4
2026-09-19 22:06:47 +0100 - dagster - DEBUG - continuous_training_job - 17040a20-829a-4933-90ef-6cbf9fb76a0c - 704329 - transform - STEP_OUTPUT - Yielded output "result" of type "Nothing". (Type check passed).
2026-09-19 22:06:47 +0100 - dagster - DEBUG - continuous_training_job - 17040a20-829a-4933-90ef-6cbf9fb76a0c - 704329 - transform - STEP_SUCCESS - Finished execution of step "transform" in 6.18s.
2026-09-19 22:06:47 +0100 - dagster - DEBUG - continuous_training_job - 17040a20-829a-4933-90ef-6cbf9fb76a0c - 701166 - test_data - STEP_WORKER_STARTING - Launching subprocess for "test_data".
2026-09-19 22:06:48 +0100 - dagster - DEBUG - continuous_training_job - 17040a20-829a-4933-90ef-6cbf9fb76a0c - 704573 - test_data - STEP_WORKER_STARTED

```

Figure 45 : Job en cours : ingestion, validation, transformation et tests terminés, entraînement en cours ; le journal d'exécution est déplié (instance locale de démonstration).

DERNIER JOB

Terminé

Simulation · 19/09/2026 22:04:38 · anass

Ingestion

Validation

Transformation

Tests

Entraînement

Contrôle de dérive

Registry

► Journal d'exécution

MODÈLE : PRODUCTION ET CANDIDAT

	PRODUCTION ACTUELLE	CANDIDAT (STAGING)	ÉCART
Version	v2	v3	
PR-AUC	0,8444	0,6771	-0.1673
ROC-AUC	0,9761	0,9568	-0.0193
F1	0,8290	0,7761	-0.0529
Précision	0,8421	0,8298	-0.0123
Rappel	0,8163	0,7290	-0.0874

Enregistrer & déployer...

Rejeter le candidat

Figure 46 : Job terminé (sept étapes en succès) et comparaison entre la Production actuelle (v2) et le candidat en Staging (v3) ; les écarts négatifs sont affichés en rouge.

Dérive (Figures 47 et 48). Après la simulation, la page Dérive affiche un bandeau rouge (« PSI max 0,489 $\geq$ seuil 0,25 ») : les cartes Montant et Log du montant sont en zone rouge, l'heure de la journée reste stable (0,003). « Montant » et « Log du montant » ont le même PSI, le logarithme étant monotone. L'évolution du PSI et la distribution par déciles de la référence (Figure 47) montrent où les données récentes s'écartent. À titre de comparaison, la même page sur Komodo (Figure 48) est stable (PSI 0,003) :

il y a moins de 100 lignes nouvelles, la référence et le récent sont donc les deux moitiés temporelles du dataset (142 403 et 142 404 lignes).



Figure 47 : Page Dérive après la simulation de 3 000 lignes à dérive forte : alerte rouge, PSI de 0,489 sur le montant, évolution du PSI et distribution du montant (instance locale de démonstration).

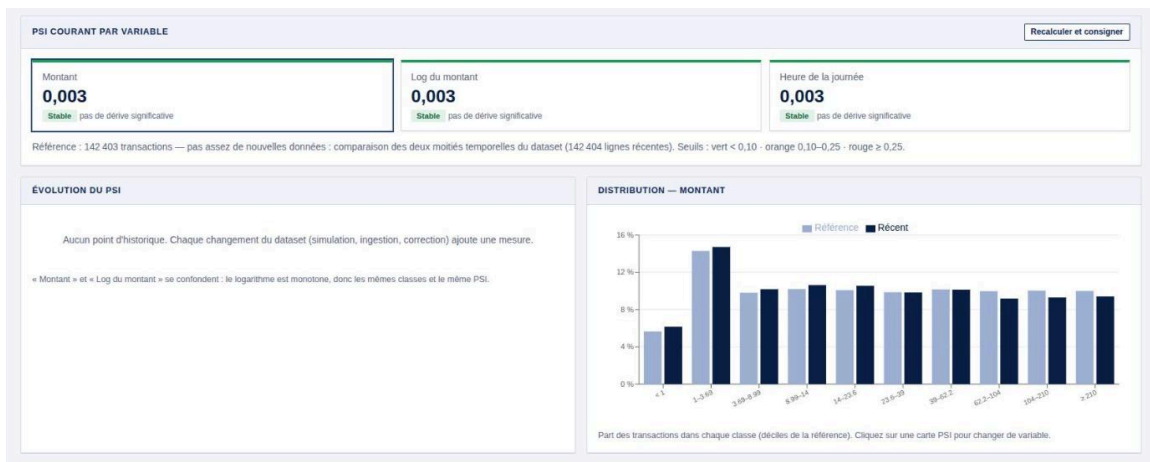


Figure 48 : Page Dérive de l'instance Komodo : PSI de 0,003 sur les trois variables, aucune dérive significative ; le graphe d'évolution est vide tant qu'aucune mesure n'a été consignée.

Approbation du modèle (Figure 49). Le bouton « Enregistrer & déployer... » du panneau « Modèle : Production et candidat » ouvre une confirmation qui montre l'avant et l'après (Figure 49) et rappelle la conséquence : promotion en Production dans le registre MLflow, export de model.pkl, puis redémarrage du service de scoring. Ce candidat a été rejeté (« Candidat rejeté — la version reste archivée dans le

registre ») : la Production reste la version 2. Le redémarrage passe par l'API Komodo (RestartStack) ; cet appel n'a pas pu être testé hors Komodo (Chapitre 8).

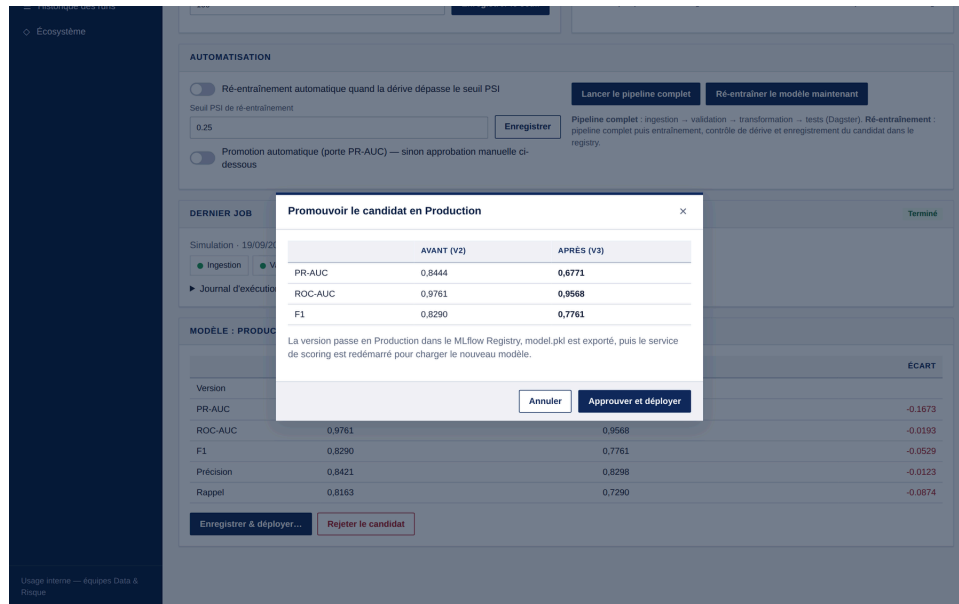


Figure 49 : Confirmation « Promouvoir le candidat en Production » : PR-AUC 0,8444 avant, 0,6771 après ; l'utilisateur peut annuler ou approuver (instance locale de démonstration).

Versions et rollback (Figures 50 et 51). Chaque changement du dataset est une version, c'est-à-dire un commit git sur le pointeur .dvc. La restauration (Figure 50) signale que les lignes en attente sont d'abord enregistrées dans une version dédiée et qu'elle est impossible pendant qu'un job tourne (HTTP 409). Elle crée un nouveau commit « Rollback dataset to 50bbdc9d » (Figure 51) : le dataset revient à 284 807 lignes, sans réécriture de l'historique, et les versions antérieures restent disponibles.

The screenshot shows the 'Versions du dataset' page with a confirmation dialog box titled 'Confirmer le rollback'. The dialog contains the following text:

Le dataset sera restauré à l'état de la version 50bbdc9d (19/09/2026 22:04:23).
Les lignes en attente non encore versionnées sont d'abord enregistrées dans une version dédiée : rien n'est perdu. Impossible si un job du pipeline est en cours.

Buttons: Annuler, Restaurer cette version

COMMIT	HORODATAGE	AUTEUR	TYPE	LIGNES MODIFIÉES	TOTAL LIGNES	RÉSUMÉ
92381438	19/09/2026 22:04:38	anass via Fraud Pipeline Bot	Simulation	3 000	287 807	data: +3000 transactions (simulate) — seuil de 100 atteint
50bbdc9d	19/09/2026 22:04:23	anass via Fraud Pipeline Bot	Correction manuelle	1	284 807	Manual correction: 1 row updated — via dashboard, user anass, 2026-09-19T21:04:21.256660+00:00
6ddc5722	18/09/2026 10:51:41	Yassin Farhi	—	—	—	feat: version raw dataset with DVC, add MiniIO as shared remote

Figure 50 : Confirmation de rollback vers la version 50bbdc9d (correction manuelle d'une ligne) (instance locale de démonstration).

The screenshot shows the 'Versions du dataset' page after the rollback. The current version is 51b42bb7, which is a rollback to version 50bbdc9d. The table shows the following versions:

COMMIT	HORODATAGE	AUTEUR	TYPE	LIGNES MODIFIÉES	TOTAL LIGNES	RÉSUMÉ
51b42bb7	19/09/2026 22:21:42	anass via Fraud Pipeline Bot	Rollback	—	284 807	Rollback dataset to 50bbdc9d — via dashboard, user anass, 2026-09-19T21:21:41.401561+00:00
92381438	19/09/2026 22:04:38	anass via Fraud Pipeline Bot	Simulation	3 000	287 807	data: +3000 transactions (simulate) — seuil de 100 atteint
50bbdc9d	19/09/2026 22:04:23	anass via Fraud Pipeline Bot	Correction manuelle	1	284 807	Manual correction: 1 row updated — via dashboard, user anass, 2026-09-19T21:04:21.256660+00:00
6ddc5722	18/09/2026 10:51:41	Yassin Farhi	—	—	—	feat: version raw dataset with DVC, add MiniIO as shared remote

Figure 51 : Versions du dataset après le rollback : commit 51b42bb7 « Rollback », version courante à 284 807 lignes, au-dessus de la simulation (+3 000 lignes), de la correction manuelle et du versionnement initial de Yassin Farhi.

Historique des runs et écosystème (Figures 52 et 53). L'historique liste chaque run (déclencheur, lignes, statut, empreinte DVC, commit) ; une ligne dépliée montre les étapes et le journal, et « Relancer » rejoue le job (Figure 52 : run « Relance » réussi sur l'instance locale). La page Écosystème de Komodo (Figure 53) vérifie depuis le serveur de la plateforme (réseau interne Docker), toutes les 30 secondes, la disponibilité de MLflow, Grafana, Prometheus, de l'API de scoring, de MinIO, de Dagster et de la documentation Swagger de platform-api : les sept sont « disponibles », avec des latences de 4 à 7 ms.

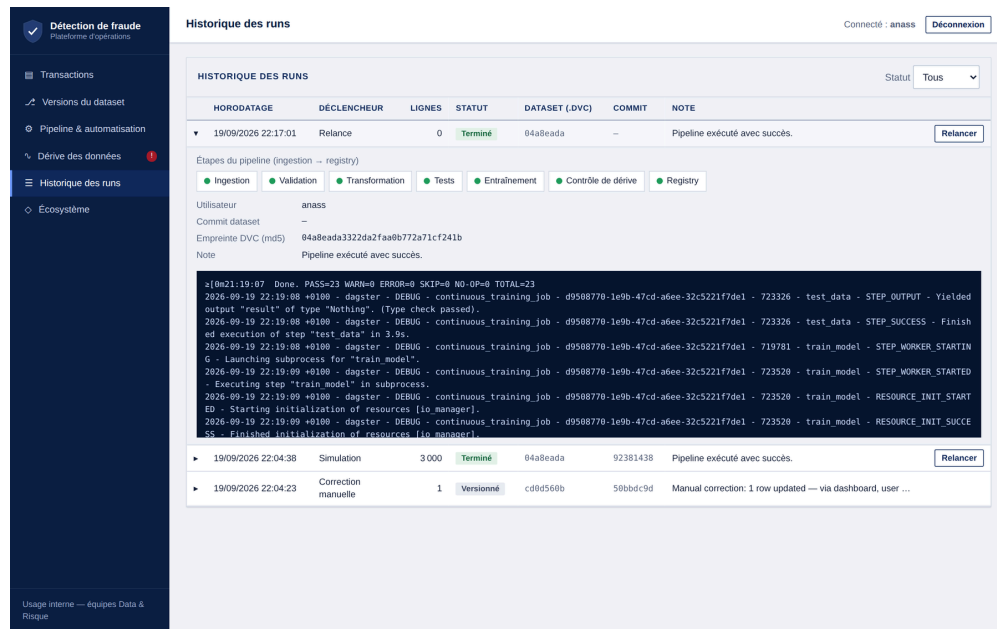


Figure 52 : Historique des runs : le run de relance est déplié (étapes en succès, empreinte DVC, journal), avec les entrées Simulation et Correction manuelle (instance locale de démonstration).

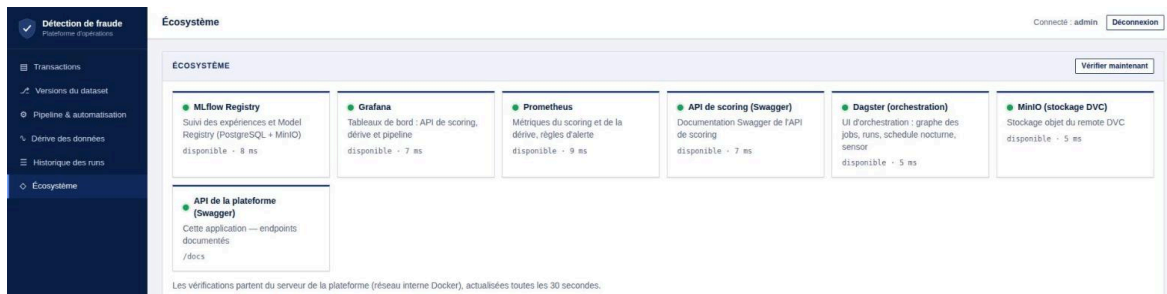


Figure 53 : Page Écosystème de l'instance Komodo (22 septembre 2026) : MLflow, Grafana, Prometheus, API de scoring, Dagster, MinIO et API de la plateforme — sept services, tous « disponibles ».

La page Pipeline & automatisation de la même instance (Figure 60) confirme que la promotion automatique reste désactivée (basculer de gauche) et ajoute un lien « Ouvrir l'UI Dagster » vers l'interface d'orchestration, détaillée au paragraphe suivant ; le dernier job (ré-entraînement manuel du 22 septembre 2026, admin) s'est terminé avec ses sept étapes en succès (ingestion, validation, transformation, tests, entraînement, contrôle de dérive, registry).

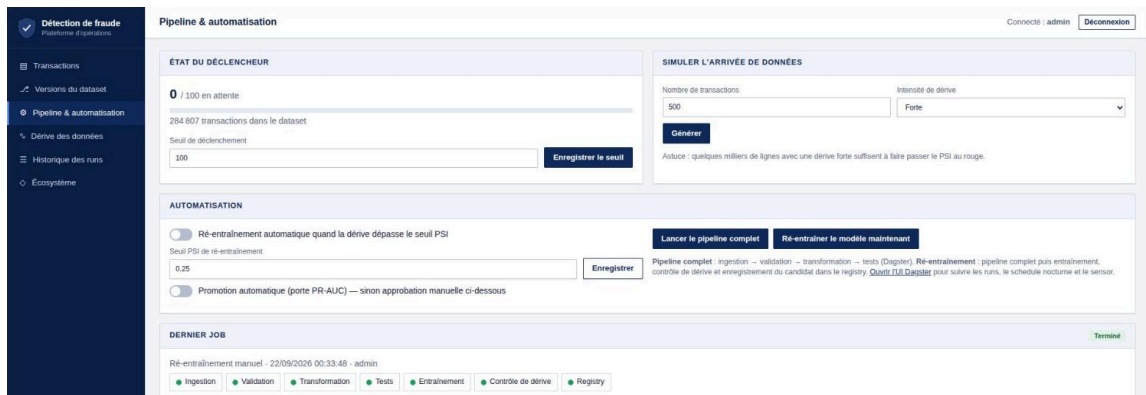


Figure 60 : Page Pipeline & automatization de l'instance Komodo : dernier job terminé (sept étapes), promotion automatique désactivée, lien « Ouvrir l'UI Dagster ».

9. Orchestration Dagster, MLflow serveur et supervision de la dérive :

Les figures suivantes proviennent toutes de l'instance déployée sur Komodo (exp.s3.fsbm.ma), consultée en lecture seule le 22 septembre 2026.

Automatisation Dagster (Figure 54). La page Automation liste les deux définitions actives de job.py : le sensor dataset_version_sensor et le schedule nightly_pipeline_schedule, programmé à 02:00 (fuseau GMT+1, Africa/Casablanca).

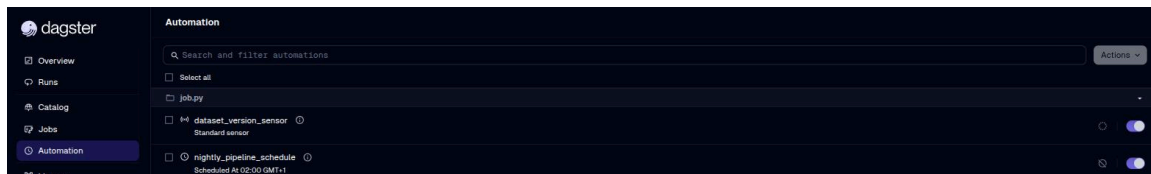


Figure 54 : Automation de l'UI Dagster : sensor `dataset\_version\_sensor` (actif) et schedule `nightly\_pipeline\_schedule` à 02:00 GMT+1.

Un run réel (Figure 55). Le run continuous_training_job déclenché depuis le tableau de bord apparaît dans l'UI Dagster avec le statut Success et une durée de 4 min 26 s ; son identifiant (6b86da83) est celui affiché dans le journal d'exécution de la plateforme (Figure 60).



Figure 55 : Run `continuous\_training\_job` dans l'UI Dagster : statut Success, durée 0:04:26, lancé manuellement depuis le tableau de bord.

Registre neuf (Figure 56). Le modèle enregistré par ce run, fraud-detection-classifier version 1, porte le statut « Stage (deprecated) : Staging » dans l'interface MLflow : la promotion automatique étant

désactivée, ce candidat attend une approbation humaine avant de remplacer le modèle actuellement servi.

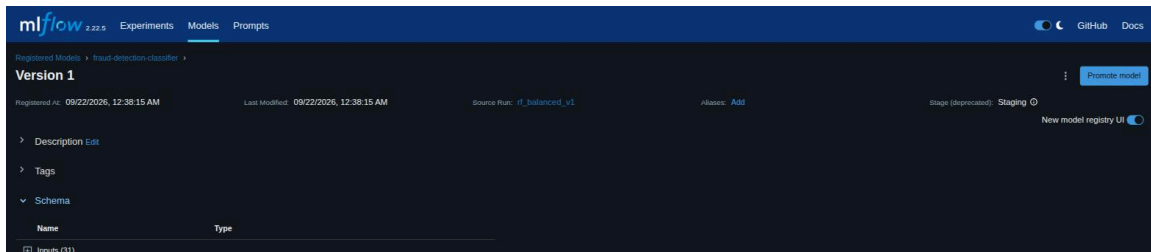


Figure 56 : `fraud-detection-classifier` version 1 dans l'interface MLflow : Stage (deprecated) : Staging.

Alertes de dérive et de pipeline (Figures 57 et 58). Le tableau de bord Grafana « Dérive & pipeline » (Figure 57) affiche un PSI stable (0,003), 284 807 lignes, le dernier run en SUCCÈS et aucune mise à jour en attente ; le panneau Prometheus /alerts (Figure 58) confirme que les sept règles d'alerte sont chargées et toutes inactives (aucune dérive ni panne en cours).

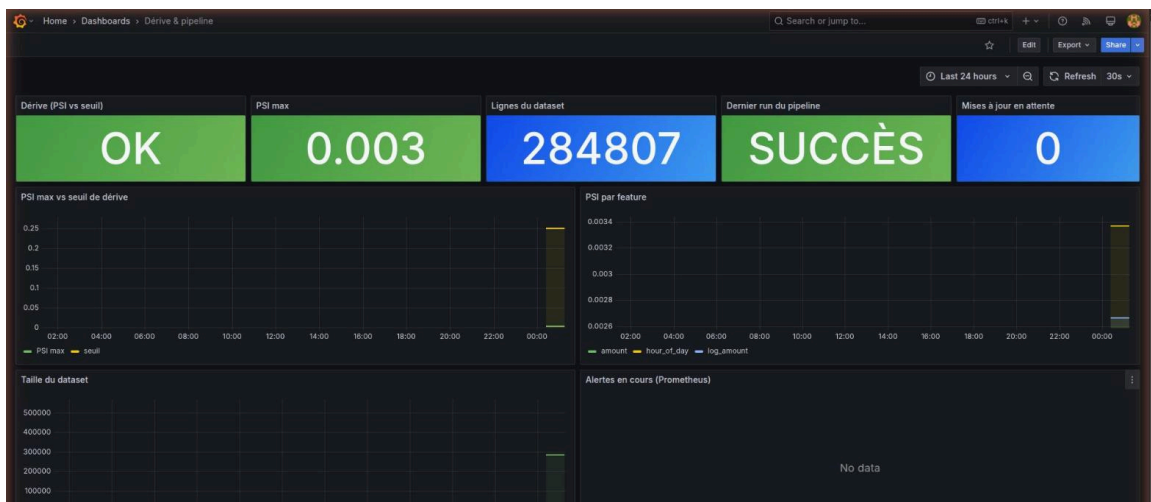


Figure 57 : Tableau de bord Grafana « Dérive & pipeline » : PSI OK (0,003), 284 807 lignes, dernier run SUCCÈS.

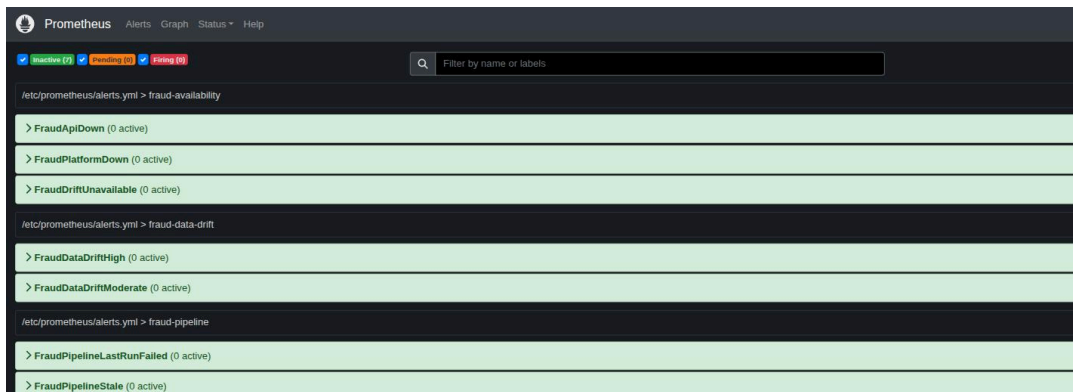


Figure 58 : Interface Prometheus /alerts : sept règles (disponibilité, dérive, pipeline), toutes Inactive.

10. Déploiement continu gardé par la CI :

Le webhook GitHub → Komodo historique déployait à chaque push, y compris quand la CI était rouge. Le job deploy de `.github/workflows/ci.yml` (needs: test, uniquement sur push vers main) appelle désormais `scripts/komodo_deploy.py`, qui lance DeployStack via l'API Komodo, attend le statut de la mise à jour puis effectue un test de santé optionnel : un push dont les tests échouent n'atteint donc plus Komodo. Trois secrets GitHub (`KOMODO_URL`, `KOMODO_API_KEY`, `KOMODO_API_SECRET`) et le nom de la stack (`KOMODO_STACK`) portent ces appels ; s'ils sont absents (fork, premier push), le job affiche un avertissement et se termine sans échec.

Le serveur `vh3` étant mutualisé, l'accès aux registres d'images (`quay.io`) échoue parfois de façon transitoire (« TLS handshake timeout ») pendant l'étape `Compose Pull` ; Komodo abandonne alors avant de remplacer un conteneur, si bien que l'ancienne pile continue de tourner et seul le job GitHub passe au rouge. `komodo_deploy.py` relance désormais ces erreurs réseau jusqu'à trois fois, à une minute d'intervalle, et échoue immédiatement sur une erreur réelle (port déjà utilisé, échec de build) plutôt que de retenter inutilement.

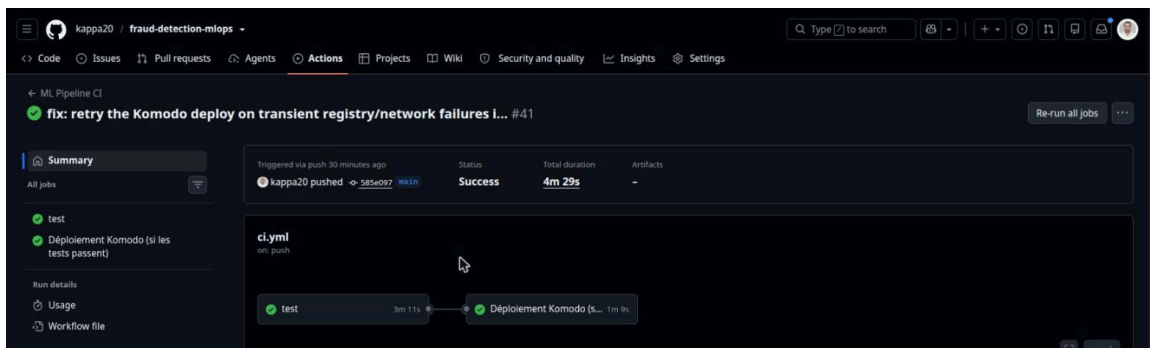


Figure 59 : Exécution GitHub Actions du commit 585e097 : le job test (3 min 11 s) précède et conditionne le job « Déploiement Komodo (si les tests passent) » (1 min 9 s), tous deux en succès.

Chapitre 7 : Exemples concrets d'ingénierie MLOps rencontrés pendant le projet

Exemple 1 — La reproductibilité des chemins (dlt).

En validant la CI/CD dans un clone Git isolé — plutôt que de compter sur GitHub Actions pour découvrir les problèmes — l'équipe a détecté un vrai bug de reproductibilité : dlt mémorise localement (`~/dlt/pipelines/<nom>/`) le répertoire de travail du **premier** run et résout les chemins relatifs des runs suivants par rapport à celui-ci, même depuis un autre répertoire ou une autre machine. Une ingestion relancée en CI aurait donc pu écrire silencieusement dans le mauvais fichier.

Le problème a été corrigé en résolvant systématiquement le chemin de destination en chemin absolu (commit fa113a8), puis revérifié avec succès. C'est une illustration directe d'un principe vu au Chapitre 2 du cours : « Ça marche sur mon laptop, ça marche pas sur mon serveur — l'absence d'environnement maîtrisé est la première cause d'irreproductibilité dans les projets data/ML. »

Exemple 2 — Exécuter la boucle complète plutôt que se contenter de tests unitaires. Le premier passage réel de la boucle plateforme → DVC → Dagster → MLflow sur les 284 807 lignes a fait apparaître trois défauts qu'aucun test avec mocks n'avait révélés :

- Le point d'entrée du conteneur (Dockerfile.platform) générait `.dvc/config.local` par un `printf` d'une ligne mal échappé : DVC rejetait le fichier. Il a été remplacé par un véritable script shell (docker/platform-entrpoint.sh), plus lisible et moins fragile.
- `dvc-s3` n'était pas dans `requirements.txt` : `dvc push` échouait avec `No module named 'dvc_s3'` dans tout environnement neuf.
- En cas d'échec du versioning, le compteur de mises à jour en attente était tout de même remis à zéro, ce qui faisait disparaître silencieusement des lignes jamais versionnées. Il n'est désormais réinitialisé qu'en cas de succès.

Un quatrième défaut, lui, a été attrapé par un test de la porte de promotion : `get_latest_versions` levait une exception au lieu de signaler « aucun modèle en Production » lors du tout premier run. Ce cas est désormais géré.

Exemple 3 — Deux frictions d'infrastructure. Les images MinIO du Docker Hub exigent désormais une connexion (changement de licence) : l'image officielle est donc tirée depuis quay.io, avec une version épinglée. Par ailleurs, une base `mlflow.db` créée par une version de MLflow plus ancienne que celle épinglée (2.22.5) provoque l'erreur `Detected out-of-date database schema` ; la parade (sauvegarde, puis `mlflow db upgrade`, migration additive) est documentée dans `docs/03_installation.md`.

Exemple 4 — Un premier run sur un registre vide. La démonstration du 19 septembre 2026 a montré qu'un registre MLflow vide sur Komodo (`mlflow.db` non versionné) fait promouvoir automatiquement le premier candidat, ici moins bon que le modèle historique, et que la republication de l'instantané

mlflow_snapshot/ a remplacé l'historique affiché par l'interface MLflow déployée (versions 1 et 2 de juillet remplacées par la seule version 1 du run du jour). Leçon : l'état d'un registre est une donnée à versionner ou à initialiser explicitement, au même titre que le dataset.

Exemple 5 — Un registre vide ne doit pas être une panne. Le panneau de comparaison Production / candidat interrogeait le registre MLflow et renvoyait une erreur 503 lorsque le registre était vide, ce qui affichait une erreur au lieu de « aucun candidat en attente ». Le correctif (commit d597ed0) traite un registre neuf comme l'absence de candidat, avec un test dédié. Il ne modifie pas la règle de promotion de l'Exemple 4, qui reste une limite documentée (Chapitre 8).

Exemple 6 — Une base MLflow plus récente que la bibliothèque. En préparant l'instance locale de démonstration, le panneau « Modèle » affichait « Detected out-of-date database schema ». La base mlflow.db du dépôt avait été migrée par la version épinglée dans requirements.txt (mlflow 2.22.5), alors que l'environnement Python de la machine de développement utilisait mlflow 2.17.2 ; la commande mlflow db upgrade ne peut pas ramener une base vers un schéma plus ancien. La sauvegarde réalisée avant la migration (mêmes versions de modèle) a été copiée dans le clone jetable, sans toucher au dépôt. Leçon : aligner l'environnement sur requirements.txt avant d'exécuter le pipeline.

Exemple 7 — Décider qu'un serveur MLflow est vivant depuis le schéma de l'URI, pas depuis la variable d'environnement. Le code qui choisissait entre le mode local (SQLite) et un serveur MLflow vivant se fiait à la présence de la variable MLFLOW_TRACKING_URI. Or mlflow.tracking.set_tracking_uri(), depuis la version 2.9, écrit lui-même cette variable dans l'environnement du processus — y compris en mode local, où elle pointe vers un fichier SQLite. Cette confusion a cassé la CI une fois : le mode « serveur vivant » (qui saute la republication de l'instantané) se déclenchait aussi en local. Le correctif (commit afad609) décide désormais depuis le schéma de l'URI (http/https) plutôt que depuis la présence de la variable, avec deux tests dédiés.

Exemple 8 — Un déploiement qui échoue sans casser le service. Le serveur Komodo (vh3) est mutualisé avec d'autres projets ; ses accès aux registres d'images (quay.io) échouent parfois de façon transitoire pendant l'étape Compose Pull, avant que Komodo n'ait remplacé le moindre conteneur — la pile en cours d'exécution n'est donc jamais interrompue par un déploiement raté, seul le job GitHub Actions passe au rouge. scripts/komodo_deploy.py distingue désormais ces erreurs réseau transitoires, relancées jusqu'à trois fois à une minute d'intervalle, des erreurs réelles (port déjà alloué, échec de build), qui font échouer le job immédiatement plutôt que de retenter inutilement (commit 585e097).

Exemple 9 — Ne pas confondre « jamais lancé » et « en échec ». Sur le tableau de bord Grafana « Dérive & pipeline », la vignette « Dernier run du pipeline » affichait un panneau rouge « No data » avant le tout premier run de la plateforme — visuellement indiscernable d'un échec réel, alors qu'aucun run n'avait encore eu lieu. Le correctif (commit fab7eaa) affiche désormais un état neutre « AUCUN RUN » dans ce cas, réservant le rouge aux échecs effectifs.

Chapitre 8 : Limites assumées et perspectives

L'équipe a choisi de documenter honnêtement les limites du travail réalisé plutôt que de les masquer :

- **Colonnes anonymisées (V1-V28)** : aucune règle métier n'a pu leur être appliquée, ces composantes PCA étant opaques (docs/data/data_quality_report.md).
- **Dimensions Fraîcheur et Intégrité non applicables** : le dataset est historique et statique, avec un schéma à une seule table de faits.
- **PSI peu discriminant** sur seulement environ 2 jours de données couvertes par le dataset — la technique est correcte, mais la donnée disponible limite ce qu'on peut en conclure en conditions réelles.
- **Conflit de versions protobuf** entre dbt-core et mlflow dans l'environnement partagé — sans impact observé, mais dette technique documentée (docs/02_architecture.md).
- **Identifiants MinIO par défaut** : MinIO (admin / admin12345) tourne avec ses identifiants par défaut sur un serveur mutualisé ; leur rotation est différée (ceux de Grafana ont été modifiés).
- **Versioning local uniquement** : les commits de versioning et de promotion restent dans le dépôt local du conteneur platform-api, sans git push automatique (choix de sécurité) ; publier une version sur GitHub reste une action humaine.
- **Dataset à récupérer manuellement sur Komodo** : dvc pull doit être lancé une fois par nouveau clone, le webhook ne le faisant pas.
- **MLflow en lecture seule (résolu en v4)** : jusqu'à la v3, l'interface MLflow de Komodo servait un instantané republié à chaque ré-entraînement, non un serveur de tracking vivant ; ce point n'est plus une limite depuis le passage à un serveur MLflow vivant (Chapitre 3 §6) — la limite qui lui succède ci-dessous documente ce que ce changement n'a pas résolu.
- **Dérive synthétique** : les données simulées sont des ré-échantillons volontairement perturbés des 5 000 premières lignes ; elles démontrent le mécanisme de détection, pas un comportement réel de production.
- **Registre MLflow neuf sur le serveur vivant** : les versions v1/v2 de l'ancien instantané ne sont pas migrées vers le backend PostgreSQL ; le premier ré-entraînement lancé depuis le tableau de bord enregistre donc une v1 fraîche en Staging — la promotion automatique restant désactivée (auto_promote: false), elle attend une approbation humaine avant de servir (constaté le 22 septembre 2026, Figure 56).
- **Authentification minimale** : comptes en variable d'environnement (PLATFORM_USERS), un seul niveau de droits, jeton HMAC de 8 heures sans refresh ni révocation ; le compte par défaut de démonstration doit être surchargé sur Komodo.

- **Redéploiement après approbation non validé hors Komodo** : l'appel RestartStack de l'API Komodo n'a pas pu être testé ; sans les variables KOMODO_*, la promotion MLflow a lieu mais le redémarrage est signalé « non effectué ».
- **Identifiant de transaction = position de ligne** : faute de clé métier, l'identifiant se décale après une suppression et n'est donc pas stable d'une version du dataset à l'autre ; le verrou optimiste (base_version, dataset_epoch) évite d'éditer de mauvaises lignes en refusant les lots préparés sur une version périmée, mais oblige à recharger la table.
- **Captures interactives sur instance locale** : les figures des flux qui écrivent (édition, rollback, job, approbation) viennent d'un clone jetable avec un remote DVC local ; les captures de Komodo sont en lecture seule.
- **Pas d'Alertmanager** : les alertes de dérive et de pipeline se consultent dans Prometheus (/alerts) et dans le panneau « Alertes en cours » du tableau de bord Grafana « Dérive & pipeline » ; aucune notification push (e-mail, Slack) n'est configurée.
- **Webhook GitHub à supprimer manuellement** : si le webhook historique vers Komodo n'est pas retiré côté Komodo, chaque push continue de déclencher un déploiement non gardé en parallèle du déploiement gardé par la CI, ce qui annule l'intérêt de la porte ; sa suppression n'a pas pu être vérifiée à distance.
- **Stack signalée UNHEALTHY par Komodo** : le conteneur à usage unique minio-init, qui crée les buckets MinIO puis s'arrête, reste Exited (0) après son exécution — Komodo l'interprète comme un service non sain bien que les huit services actifs tournent normalement ; cosmétique, non corrigé.
- **Pistes non implémentées**, hors scope du module et documentées comme perspectives : SMOTE pour le rééquilibrage des classes, split temporel pour l'entraînement, séparation des environnements DataOps/ML, gestion des secrets (rotation des identifiants MinIO), publication contrôlée des versions de données vers GitHub (pull request plutôt que push automatique), environnements dev/staging/prod distincts.

Conclusion générale :

Ce projet a démontré la faisabilité d'un système complet de détection de fraude bancaire, construit selon les principes DataOps et MLOps vus en cours plutôt que comme un simple exercice de modélisation. Les 10 livrables attendus par le cahier des charges ont été réalisés et vérifiés : pipeline DataOps automatisé et testé (27/27 tests dbt), modèle de classification retenu avec des performances vérifiées (PR-AUC de 0,8444), service de prédiction conteneurisé et testé (12/12 tests unitaires à l'origine, 71/71 tests automatisés dans cette version), chaîne CI/CD complète validée dans un clone isolé et gardant désormais le déploiement sur Komodo, et dispositif de monitoring opérationnel complété par des alertes de dérive et de pipeline.

Au-delà des livrables, le projet a permis à l'équipe de rencontrer et de résoudre un vrai problème d'ingénierie — un bug de reproductibilité lié à la résolution de chemins par dlt — illustrant concrètement pourquoi la maîtrise de l'environnement est centrale en MLOps. Les limites du travail (colonnes anonymisées, période courte pour la détection de dérive, dette technique sur les versions de dépendances) ont été assumées et documentées plutôt que masquées, avec des perspectives claires : rééquilibrage par SMOTE, split temporel, séparation des environnements DataOps/ML et gestion des secrets pour une mise en production réelle.

Dans sa seconde version, le projet a été prolongé par un versioning des données avec DVC et MinIO et par une plateforme d'entraînement continu dotée d'une porte de promotion, l'ensemble étant déployé sur six services Komodo et couvert par 57 tests automatisés. La version 3 y ajoute une interface d'opérations authentifiée (transactions versionnées avec rollback, pipeline suivi par étape, dérive, approbation du modèle) déployée sur Komodo. La version 4 y ajoute l'orchestration Dagster (schedule nocturne, sensor sur le dataset), un serveur MLflow vivant (PostgreSQL, artefacts MinIO), des alertes de dérive et de pipeline (Prometheus, Grafana) et un déploiement continu désormais gardé par la CI, portant la pile à huit services et la suite de tests à 71. Cette extension a fait apparaître, à l'exécution réelle, des défauts que les tests avec mocks n'avaient pas détectés (Chapitre 7), ce qui confirme l'intérêt de valider la boucle complète et pas seulement ses composants.

Références :

- [1] Atwal, H. (2019). Practical DataOps: Delivering Agile Data Science at Scale. Apress.
- [2] DataOps Manifesto. <https://dataopsmanifesto.org>
- [3] DataKitchen. DataOps Fundamentals Certification — supports de certification (certification/quiz_answers.pdf).
- [4] dlt (data load tool) Documentation. <https://dlthub.com/docs>
- [5] dbt Labs. dbt Documentation. <https://docs.getdbt.com>
- [6] Dagster Documentation. <https://docs.dagster.io>
- [7] MLflow Documentation. <https://mlflow.org/docs>
- [8] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- [9] FastAPI Documentation. <https://fastapi.tiangolo.com>
- [10] GitHub Actions Documentation. <https://docs.github.com/actions>
- [11] DVC (Data Version Control) Documentation. <https://dvc.org/doc>
- [12] MinIO Documentation. <https://min.io/docs>
- [13] Komodo — Build and deploy software on many servers. <https://komo.do>
- [14] Prometheus Documentation. <https://prometheus.io/docs>
- [15] Grafana Documentation. <https://grafana.com/docs>